

# 大数据计算及应用

---

## 推荐系统(2)

从<http://www.mmds.org>改编的幻灯片

# 100万美元的问题

**NETFLIX**

## Netflix Prize

Home Rules Leaderboard Register Update Submit Download

**NETFLIX**

Browse Recommendations Friends Queue Buy DVDs

Home Genres New Releases Previews Netflix Top 100 Crit

### Movies For You

Randy, the following movies were chosen based on your interest in:  
[Bowling for Columbine](#)  
[Carnivale: Season 1](#)  
[Fahrenheit 9/11](#)

**You really liked it...**  
Now own it for just \$5.99

**Welcome!**

The Netflix Prize seeks to substantially improve the accuracy of predictions about how much someone is going to love a movie based on their movie preferences. Improve it enough and you win one (or more) Prizes. Winning the Netflix Prize improves our ability to connect people to the movies they love.

Read the [Rules](#) to see what is required to win the Prizes. If you are interested in joining the quest, you should [register a team](#).

You should also read the [frequently-asked questions](#) about the Prize. And check out how various teams are doing on the [Leaderboard](#).

Good luck and thanks for helping!

[FAQ](#) | [Forum](#) | [Netflix Home](#)

© 1997-2006 Netflix, Inc. All rights reserved.

# Netflix奖

---

## □ 训练数据

- 1亿次评分，48万用户，17770部电影
- 6年数据：2000-2005

## □ 测试数据

- 每个用户的最后几次评分（280万） ■

**Evaluation criterion:** Root Mean Square Error (RMSE)

$$= \sqrt{\sum_{(i,x) \in R} (\hat{r}_{xi} - r_{xi})^2 / |R|}$$

- Netflix系统RMSE：0.9514

## □ 竞争

- 2,700+支队伍
  - Netflix 10%的改进奖金为100万美元
-

# Netflix实用矩阵R

---

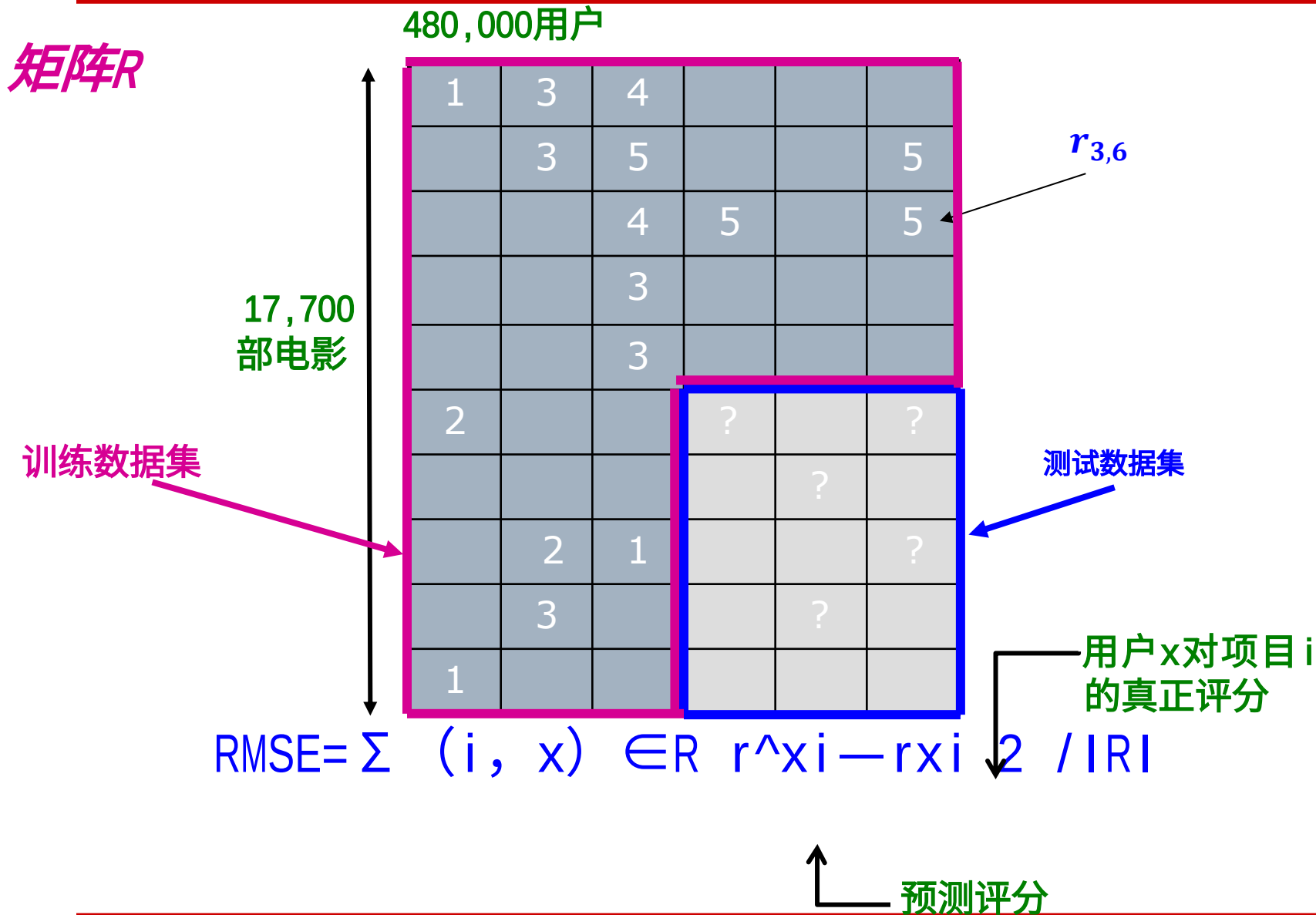
矩阵R

480,000用户

17,700部电影

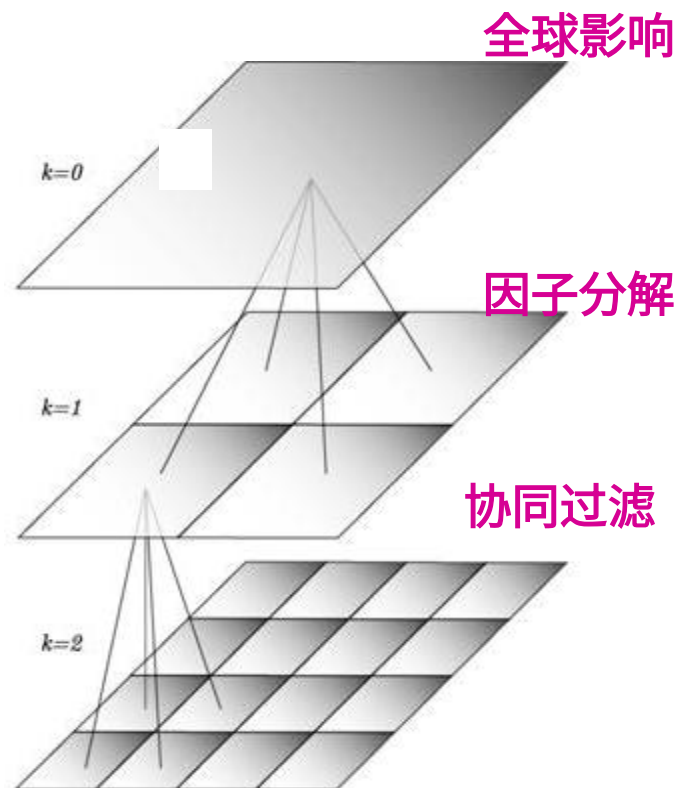
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 3 | 4 |   |   |   |
|   | 3 | 5 |   |   | 5 |
|   |   | 4 | 5 |   | 5 |
|   |   | 3 |   |   |   |
|   |   | 3 |   |   |   |
| 2 |   |   | 2 |   | 2 |
|   |   |   |   | 5 |   |
|   | 2 | 1 |   |   | 1 |
|   | 3 |   |   | 3 |   |
| 1 |   |   |   |   |   |

# 效用矩阵R：评价



# Bel I Kor推荐系统

- Netflix挑战赛的获胜者！
- 数据的多尺度建模：  
将数据的“区域”建模与  
顶层相结合  
一种精致的本地视图：
- 全球
  - 用户/电影总体偏差
- 因子分解
  - 解决“区域”影响
- 协同过滤
  - 提取本地模式



# 建模局部和全局效应

## □ 全球

- 平均电影评级：3.7颗星
- 《第六感》比平均水平高出0.5颗星。
- 乔的评分低于平均水平0.2星。

→ 基准估算：

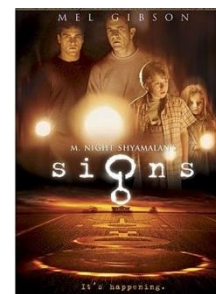
乔将《第六感》评为4星

## □ 当地社区 (CF/NN)：

- 乔不喜欢相关的电影《迹象》

→ 最终估计：

乔将《第六感》评为3.8星





# 回顾：协同过滤（CF）

---

- 最早和最流行的协同过滤方法
- 从“类似”电影的评分中推导出未知评分（项目-项目变体）
- 定义项目*i*和*j*的相似度量 $s_{ij}$
- 选择*k*近邻，计算评分

■  $N(i; x)$  : 由  $x$  评价的与  $i$  最相似的项目

$$\hat{r}_{xi} = \frac{\sum_{j \in N(i; x)} s_{ij} \cdot r_{xj}}{\sum_{j \in N(i; x)} s_{ij}}$$

$s_{ij} r_{xj} N(\dots i; i_x)$   
si  
ra  
te

milarity of items  $i$  and  $j$  is the  
 average of user  $x$  on item  $i$  and item  $j$   
 over the set of items similar to item  $i$   
 that were rated by user  $x$

# 建模局部和全局效应

- 在实践中，如果我们对偏差进行建模，我们就能得到更好的估计：

$$\hat{r}_{xi} = b_{xi} + \frac{\sum_{j \in N(i; X)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i; X)} s_{ij}}$$

$r_{xi}$ 的基线估计值

$$b_{xi} = \mu + b_x + b_i$$

$\mu$  = 总体平均评分

$b_x$  = 用户 $x$ 的评分偏差 $x$  = (用户 $x$ 的平均评分) -  $\mu$

$b_i$  = (电影 $i$ 的平均评分) -  $\mu$

问题/问题：

1) 相似度测量是“任意的”

2) 成对相似性忽略用户之间的相互依赖关系

3) 取加权平均值可能会受到限制  
解决方案：不要使用 $s_{ij}$ ，而是使用我们直接从数据中估计的 $w_{ij}$

# 想法：插值权重 $w_{ij}$

---

- 使用加权总和而不是加权平均值：

$$\widehat{r_{xi}} = b_{xi} + \sum_{j \in N(i;x)} w_{ij}(r_{xj} - b_{xj})$$

- 几点说明：

- $N(i; x)$  ……用户 $x$ 对与电影 $i$ 相似的电影进行评分的集合
- $w_{ij}$ 是插值权重（某个实数）
  - 我们允许： $\sum_{j \in N(i, x)} w_{ij} = 1$
- $w_{ij}$ 模型用于分析电影对之间的交互作用（不依赖于用户 $x$ ）

# 想法：插值权重 $w_{ij}$

---

□  $\hat{r}_{xi} = b_{xi} + \sum_{j \in N(i, x)} w_{ij} (r_{xj} - b_{xj})$

□ 如何设置  $w_{ij}$  ?



Remember, error metric is:  $\sqrt{\sum_{(i,x) \in R} (\hat{r}_{xi} - r_{xi})^2 / |R|}$  or  
等价于SSE:  $\sum_{(i,x) \in R} (\hat{r}_{xi} - r_{xi})^2$

■ 找到最小化训练数据SSE的  $w$  !

□ 模型中包含项目  $i$  与其邻近项目  $j$  之间的关系

■  $w_{ij}$  可以根据  $x$  和所有其他对  $i$  进行评分的用户来学习/估计

**为什么这是一个好主意？**



# 通过优化进行的建议

## □ 目标：提出好的建议

- 使用RMSE量化良好性：  
较低的RMSE→意味着更好的建议
- 想要对用户尚未看到的项目提出好的建议。  
无法做到这一点！
- 让我们建立一个系统，使其在已知的（用户，项目）评级上运行良好  
希望该系统也能很好地预测未知的评级

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 3 | 4 |   |   |   |
|   | 3 | 5 |   |   | 5 |
|   |   | 4 | 5 |   | 5 |
|   |   | 3 |   |   |   |
|   |   | 3 |   |   |   |
| 2 |   |   | 2 |   | 2 |
|   |   |   |   | 5 |   |
|   | 2 | 1 |   |   | 1 |
|   | 3 |   |   | 3 |   |
| 1 |   |   |   |   |   |

# 通过优化进行的建议

---

- 想法：让我们设定值 $w$ ，使它们在已知的（用户，项目）评级上工作良好
- 如何找到这样的值 $w$ ？
- 想法：定义目标函数，求解优化问题

- 找到最小化训练数据SSE的 $w$ ！

$$J(w) = \sum_{x,i} \left( \underbrace{\left[ b_{xi} + \sum_{j \in N(i;x)} w_{ij} (r_{xj} - b_{xj}) \right]}_{\text{预测评分}} - \underbrace{r_{xi}}_{\text{True 等级}} \right)^2$$

- 将 $w$ 视为一个实数向量

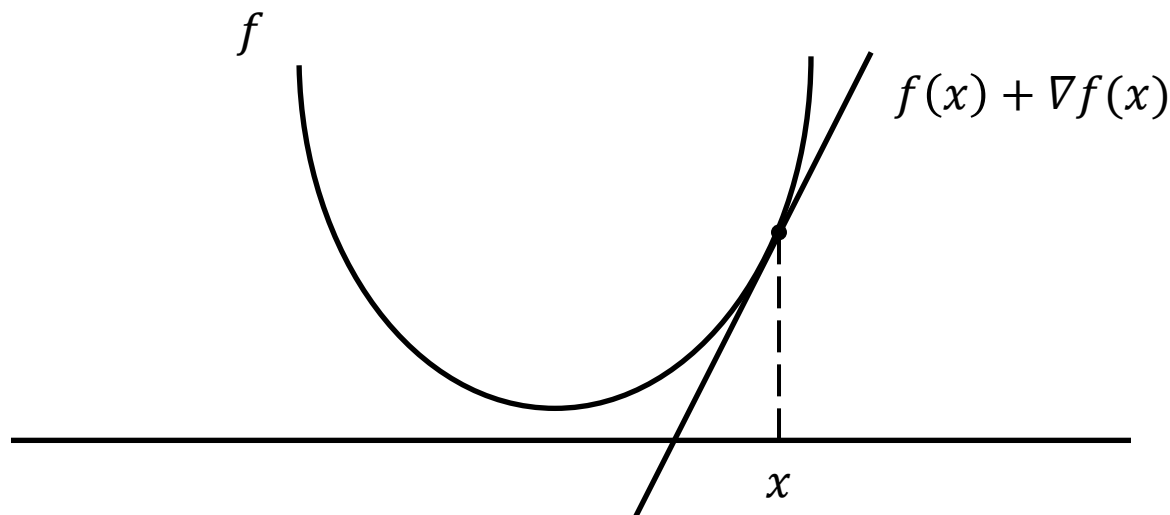


# 迂回：最小化函数

---

## □ 最小化函数 $f(x)$ 的一种简单方法：

- 计算导数 $\nabla f$
- 从某个点 $x$ 开始，计算 $\nabla f(x)$
- 在梯度的反方向上迈出一步： $x = x - \nabla f(x)$
- 重复直至收敛



# 插值权重

□ We have the optimization

$$J(w) = \sum_x \left( \left[ b_{xi} + \sum_{j \in N(i;x)} w_{ij} (r_{xj} - b_{xj}) \right] - r_{xi} \right)^2$$

问题，现在怎么办？

□ 梯度下降：

$\eta$  ……学习率

■ 迭代直至收敛： $W \leftarrow W - \eta \nabla W J$

■ 其中 $\nabla W J$ 是梯度（在数据上评估的导数）：

$$\nabla_w J = \left[ \frac{\partial J(w)}{\partial w_{ij}} \right] = 2 \sum_{x,i} \left( \left[ b_{xi} + \sum_{k \in N(i;x)} w_{ik} (r_{xk} - b_{xk}) \right] - r_{xi} \right) (r_{xj} - b_{xj})$$

for  $j \in \{N(i; x), \forall i, \forall x\}$

else  $\frac{\partial J(w)}{\partial w_{ij}} = 0$

■ 注：我们固定电影 $i$ ，遍历所有的 $r_{xi}$ ，对于每一个电影 $j \in N(i); x$ ，我们计算 $\frac{\partial J(w)}{\partial w_{ij}}$

而旧的 $|w_{new} - w_{old}| > \epsilon$ ：

$$w_{new} = w_{old} - \eta \cdot \nabla w_{old}$$

# 插值权重

□ 到目前为止： $r_{xi} = b_{xi} + \sigma_{j \in N(i; x)} (r_{xj} - b_x)$

$w_{ij}$

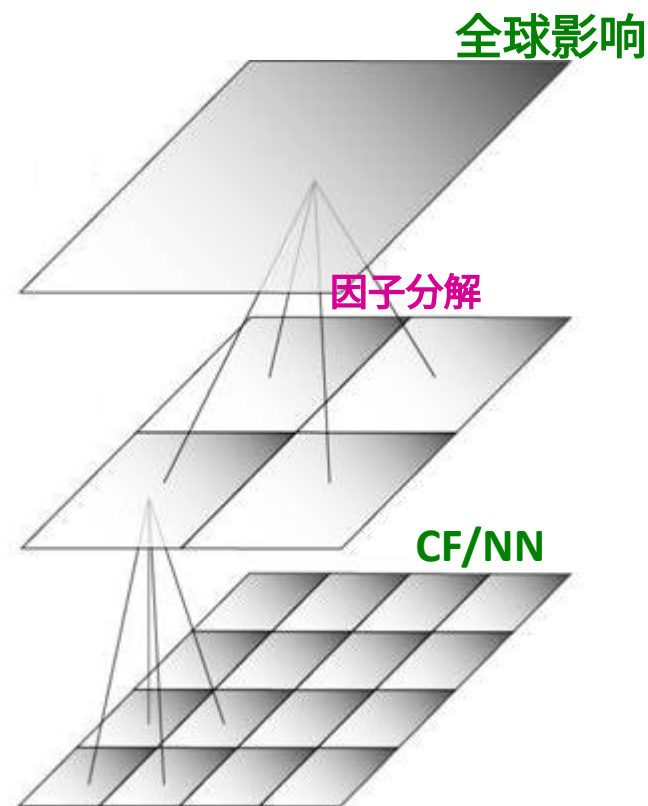
- 根据其作用推导出的权重；不使用任意相似性度量

$(w_{ij} = s_{ij})$

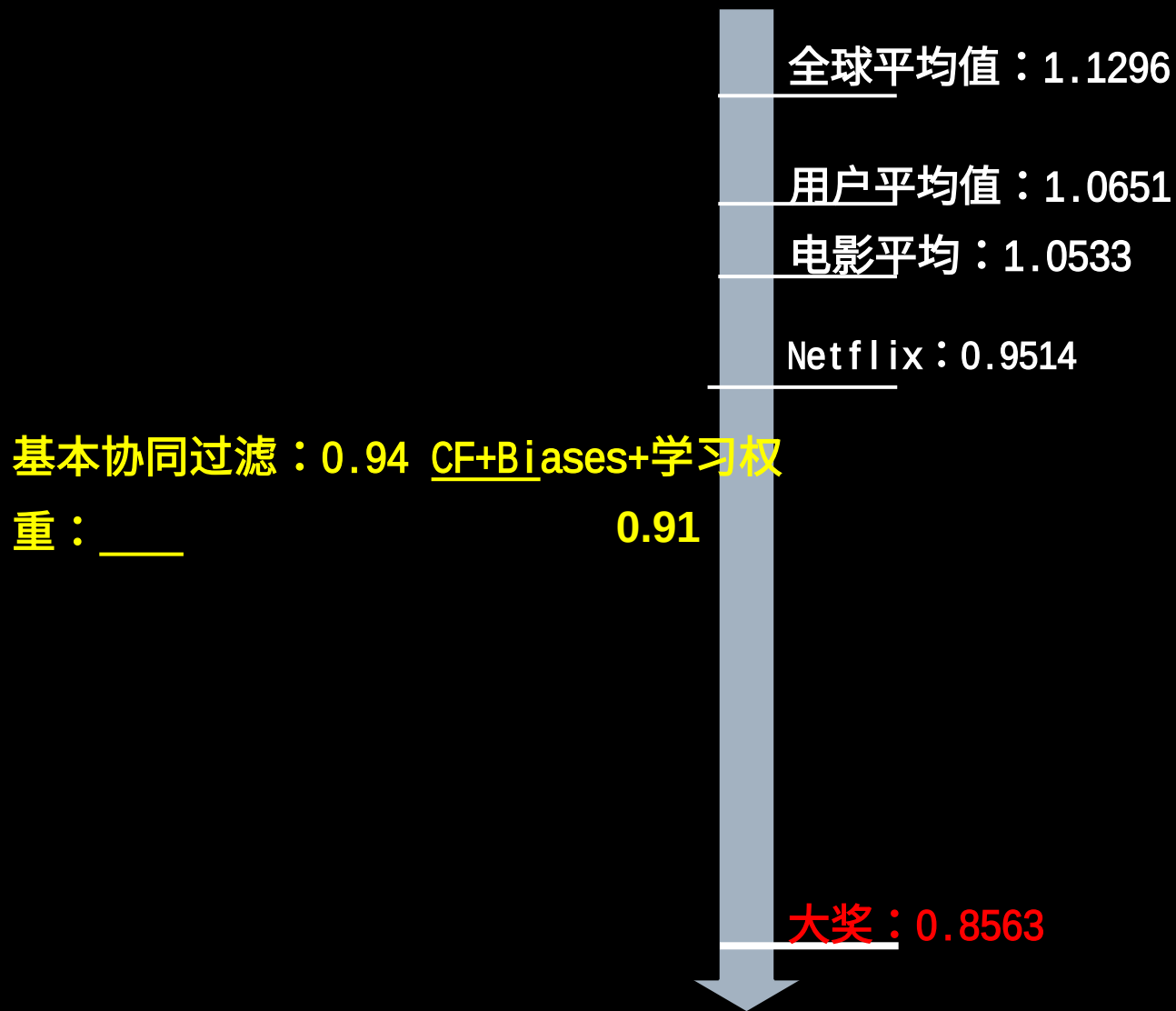
- 明确考虑相邻电影之间的相互关系

□ 下一项：潜在因素模型

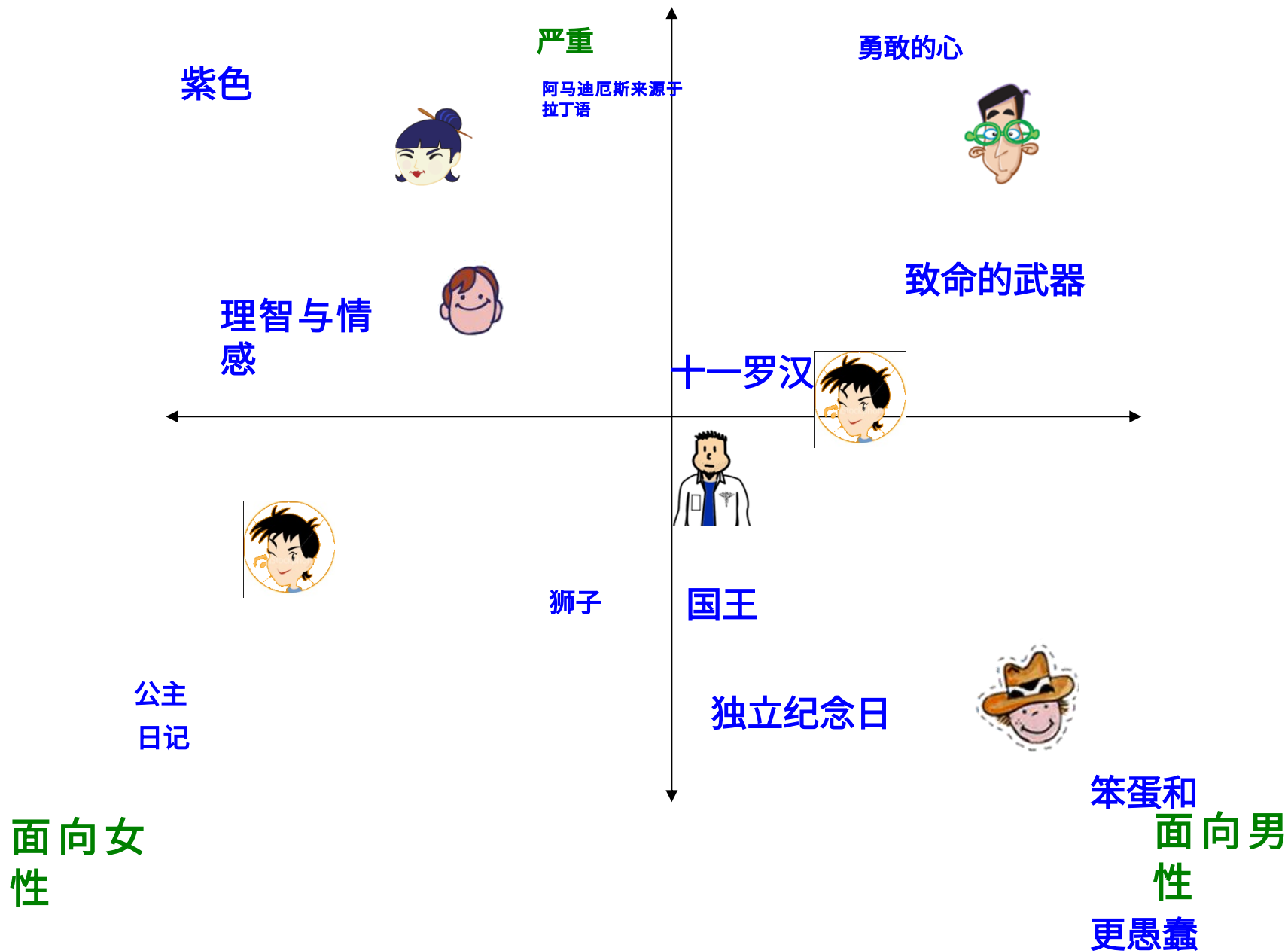
- 提取“区域”相关性



# 各种方法的性能



# 潜在因素模型（例如，SVD）





# 潜在因素模型

SVD := U  $\Sigma$  V<sup>T</sup>

□ Netflix数据上的“SVD”： $R \approx Q \cdot P^T$

用户

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 |   | 3 |   |   | 5 |   |   | 5 |   | 4 |   |
|   |   | 5 | 4 |   |   | 4 |   |   | 2 | 1 | 3 |
| 2 | 4 |   | 1 | 2 |   | 3 |   | 4 | 3 | 5 |   |
|   | 2 | 4 |   | 5 |   |   | 4 |   |   | 2 |   |
|   |   | 4 | 3 | 4 | 2 |   |   |   |   | 2 | 5 |
| 1 |   | 3 |   | 3 |   |   | 2 |   |   | 4 |   |

R

因素

|     |     |    |
|-----|-----|----|
| .1  | -.4 | .2 |
| -.5 | .6  | .5 |
| -.2 | .3  | .5 |
| 1.1 | 2.1 | .3 |
| -.7 | 2.1 | -2 |
| -1  | .7  | .3 |

Q

用户

|     |     |    |     |     |     |     |     |     |     |     |              |
|-----|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|--------------|
| 1.1 | -.2 | .3 | .5  | -2  | -.5 | .8  | -.4 | .3  | 1.4 | 2.4 | factor<br>rs |
| -.8 | .7  | .5 | 1.4 | .3  | -1  | 1.4 | 2.9 | -.7 | 1.2 | -.1 |              |
| 2.1 | -.4 | .6 | 1.7 | 2.4 | .9  | -.3 | .4  | .8  | .7  | -.6 |              |

P<sup>T</sup>

项目

≈

项目

□ 现在让我们假设我们可以将评分矩阵R近似为“薄” $Q \cdot P^T$ 的乘积

■ R缺少条目，但让我们暂时忽略这一点！

□ 基本上，我们希望在已知评分上重建误差小，而对缺失评分上的值不关心

# 评级是因素的产物

□ 如何估算缺失的评分  
用户  $x_{usef}$  项目  $i$  ?

$$\hat{r}_{xi} = q_i \cdot$$

$p_x$

项目

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 |   | 3 |   |   | 5 |   |   | 5 |   | 4 |   |
|   |   | 5 | 4 | ? |   | 4 |   |   | 2 | 1 | 3 |
| 2 | 4 |   | 1 | 2 |   | 3 |   | 4 | 3 | 5 |   |
|   | 2 | 4 |   | 5 |   |   | 4 |   |   | 2 |   |
|   |   | 4 | 3 | 4 | 2 |   |   |   |   | 2 | 5 |
| 1 |   | 3 |   | 3 |   |   | 2 |   |   | 4 |   |

≈

$$= \sum_f q_{if} \cdot p_{xf}$$

Q的第i行=PT的第x列

项目

|     |     |    |
|-----|-----|----|
| .1  | -.4 | .2 |
| -.5 | .6  | .5 |
| -.2 | .3  | .5 |
| 1.1 | 2.1 | .3 |
| -.7 | 2.1 | -2 |
| -1  | .7  | .3 |

因素Q

● 因素

用户

|     |     |    |     |     |     |     |     |     |     |     |     |
|-----|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 1.1 | -.2 | .3 | .5  | -2  | -.5 | .8  | -.4 | .3  | 1.4 | 2.4 | -.9 |
| -.8 | .7  | .5 | 1.4 | .3  | -1  | 1.4 | 2.9 | -.7 | 1.2 | -.1 | 1.3 |
| 2.1 | -.4 | .6 | 1.7 | 2.4 | .9  | -.3 | .4  | .8  | .7  | -.6 | .1  |

$P^T$





# 评级是因素的产物

□ 如何估算缺失的评分  
用户  $x$  对项目  $i$  ?

$$\hat{r}_{xi} = q_i \cdot$$

$p_x$

项目

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 |   | 3 |   |   | 5 |   |   | 5 |   | 4 |   |
|   |   | 5 | 4 | ? |   | 4 |   |   | 2 | 1 | 3 |
| 2 | 4 |   | 1 | 2 |   | 3 |   | 4 | 3 | 5 |   |
|   | 2 | 4 |   | 5 |   |   | 4 |   |   | 2 |   |
|   |   | 4 | 3 | 4 | 2 |   |   |   |   | 2 | 5 |
| 1 |   | 3 |   | 3 |   |   | 2 |   |   | 4 |   |

$\approx$

$$= \sum_f q_{if} \cdot p_{xf}$$

$q=Q$  的第  $i$  行  $p_x=PT$  的第  $x$  列

项目

|     |     |    |
|-----|-----|----|
| .1  | -.4 | .2 |
| .5  | .6  | .5 |
| -.2 | .3  | .5 |
| 1.1 | 2.1 | .3 |
| -.7 | 2.1 | -2 |
| -1  | .7  | .3 |

因素  $Q$

用户

因素

|     |     |    |     |     |     |     |     |     |     |     |     |
|-----|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 1.1 | -.2 | .3 | .5  | 2   | -.5 | .8  | -.4 | .3  | 1.4 | 2.4 | -.9 |
| -.8 | .7  | .5 | 1.4 | .3  | -1  | 1.4 | 2.9 | -.7 | 1.2 | -.1 | 1.3 |
| 2.1 | -.4 | .6 | 1.7 | 2.4 | .9  | -.3 | .4  | .8  | .7  | -.6 | .1  |

$P^T$



# 评级是因素的产物

□ 如何估算缺失的评分  
用户  $x_{usef}$  对项目  $i$  ?

$$\hat{r}_{xi} = q_i \cdot$$

$p_x$

项目

|   |   |   |   |     |   |   |   |   |   |   |   |
|---|---|---|---|-----|---|---|---|---|---|---|---|
| 1 |   | 3 |   |     | 5 |   |   | 5 |   | 4 |   |
|   |   | 5 | 4 | 2.4 | 4 |   |   | 2 | 1 | 3 |   |
| 2 | 4 |   | 1 | 2   |   | 3 |   | 4 | 3 | 5 |   |
|   | 2 | 4 |   | 5   |   |   | 4 |   |   | 2 |   |
|   |   | 4 | 3 | 4   | 2 |   |   |   |   | 2 | 5 |
| 1 |   | 3 |   | 3   |   |   | 2 |   |   | 4 |   |

≈

$$= \sum_f q_{if} \cdot p_{xf}$$

$q=Q$  的第  $i$  行 =  $PT$  的第  $x$  列

项目

|     |     |    |
|-----|-----|----|
| .1  | -.4 | .2 |
| .5  | .6  | .5 |
| -.2 | .3  | .5 |
| 1.1 | 2.1 | .3 |
| -.7 | 2.1 | -2 |
| -1  | .7  | .3 |

因素  $Q$

用户

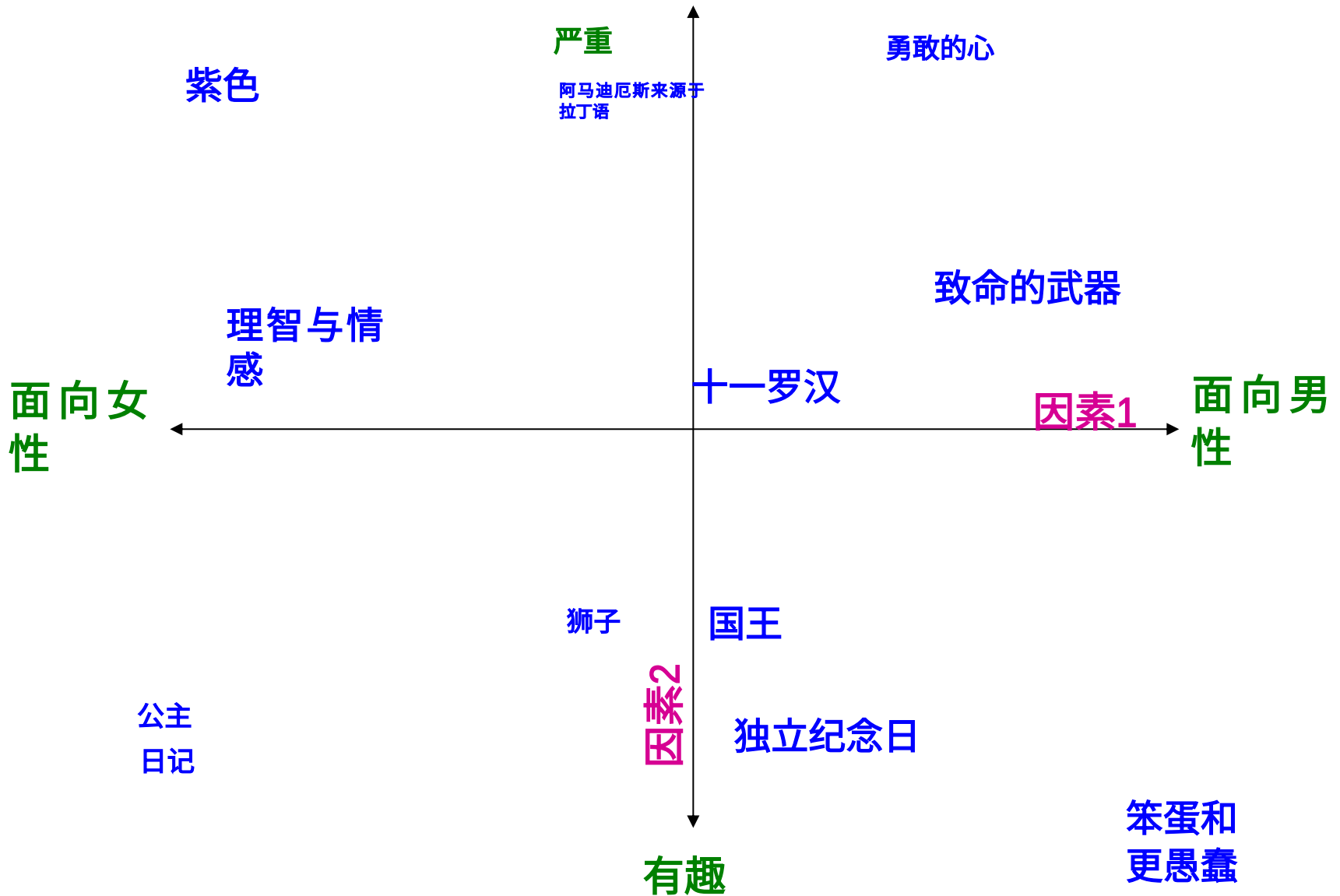
因素

|     |     |    |     |     |     |     |     |     |     |     |     |
|-----|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 1.1 | -.2 | .3 | .5  | 2   | -.5 | .8  | -.4 | .3  | 1.4 | 2.4 | -.9 |
| -.8 | .7  | .5 | 1.4 | .3  | -1  | 1.4 | 2.9 | -.7 | 1.2 | -.1 | 1.3 |
| 2.1 | -.4 | .6 | 1.7 | 2.4 | .9  | -.3 | .4  | .8  | .7  | -.6 | .1  |

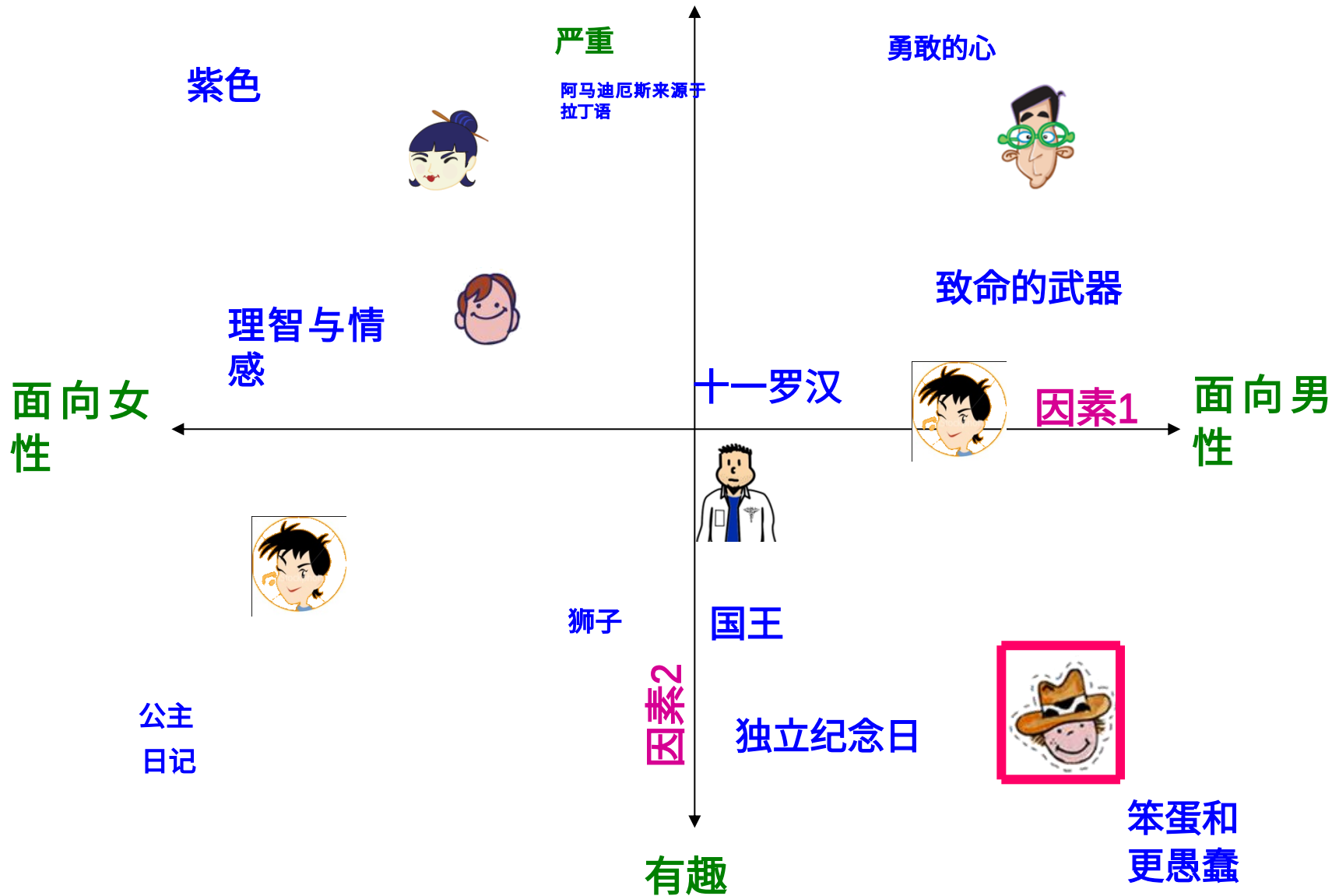
$PT$



# 潜在因素模型



# 潜在因素模型



# 总结：SVD

## □ 记住SVD：



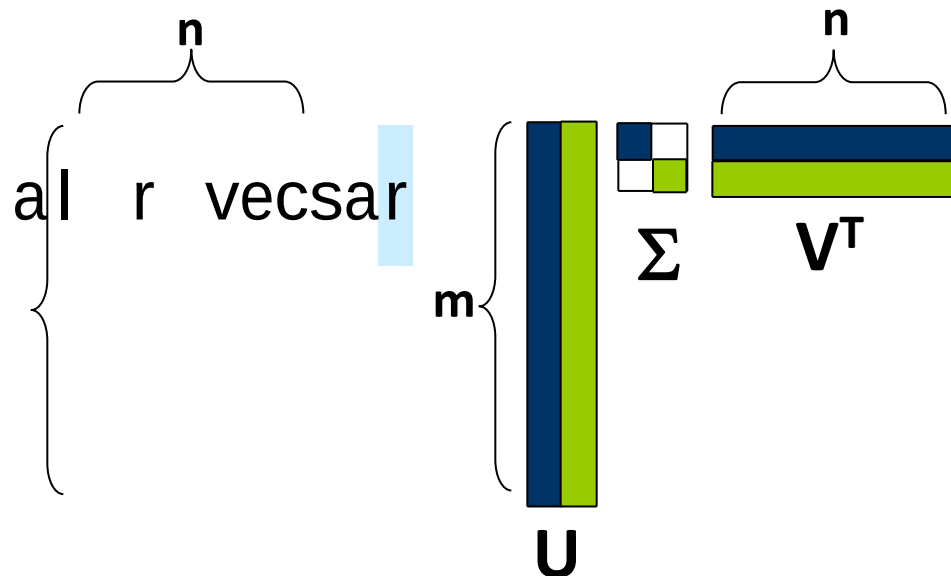
输入数据矩阵  $A$

vecs  $m$

$A \approx$



$\Sigma$ ：单值



□ 因此，在我们的情况下：

Netflix数据上的“SVD”： $R \approx Q \cdot P^T$

$$A = R, \quad Q = U, \quad P^T = \Sigma V^T$$

$$\hat{r}_{xi} = q_i \cdot p_x$$



# SVD：更多好东西

---

- 我们已经知道SVD给出了最小的重建误差（平方误差之和）：

$$\min_{U,V,\Sigma} \sum_{ij \in A} (A_{ij} - [U\Sigma V^T]_{ij})^2$$

- 注意以下两点：

- SSE和RMSE呈单调关系：

- $RMSE = \sqrt{SSE/|R|}$

好消息：SVD正在将RMSE降至最低

- 复杂性：SVD误差项中的总和是所有条目（无评级被解释为零评级）的总和。  
但是我们的R缺少条目！

# 潜在因素模型

| 用户 |   |   |   |   |   |   |   |   |   |   |   | 因素  |     |    | 用户  |     |    |     |     |     |     |     |     |     |     |     | 因素 |    |    |
|----|---|---|---|---|---|---|---|---|---|---|---|-----|-----|----|-----|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|----|----|
| 1  | 3 |   |   | 5 |   |   | 5 |   | 4 |   |   | .1  | -.4 | .2 | 1.1 | -.2 | .3 | .5  | -.2 | -.5 | .8  | -.4 | .3  | 1.4 | 2.4 | -.9 | -1 | .7 | .3 |
|    |   | 5 | 4 |   |   | 4 |   |   | 2 | 1 | 3 | -.5 | .6  | .5 | -8  | .7  | .5 | 1.4 | .3  | -1  | 1.4 | 2.9 | -.7 | 1.2 | -.1 | 1.3 | -1 | .7 | .3 |
| 2  | 4 |   | 1 | 2 |   | 3 |   | 4 | 3 | 5 |   | -.2 | .3  | .5 | 2.1 |     |    |     |     |     |     |     |     |     |     |     |    |    |    |
|    | 2 | 4 |   | 5 |   |   | 4 |   |   | 2 |   | 1.1 | 2.1 | .3 | -7  |     |    |     |     |     |     |     |     |     |     |     |    |    |    |
|    |   | 4 | 3 | 4 | 2 |   |   |   |   | 2 | 5 | -.7 | 2.1 | -2 | 2.1 |     |    |     |     |     |     |     |     |     |     |     |    |    |    |
| 1  | 3 |   | 3 |   |   | 2 |   |   | 4 |   |   | -1  | .7  | .3 | 2.1 | -.4 | .6 | 1.7 | 2.4 | .9  | -.3 | .4  | .8  | .7  | -.6 | .1  |    |    |    |

- 当条目缺失时，SVD未定义！
- 用专门的方法求出 P、Q

$$\min_{P,Q} \sum_{(i,x) \in R} (r_{xi} - q_i \cdot p_x)^2$$

$$\hat{r}_{xi} = q_i \cdot p_x$$

## 笔记

- 我们不要求P、Q的列是正交/单位长度
- P、Q将用户/电影映射到潜在空间

## □ Netflix参赛者中最受欢迎的模特

---

27

# 寻找潜在因素

# 潜在因素模型

□ 我们的目标是找到P和Q，使得：

最小值P、Q

$$\sum_i r_i (x_i - q_i \cdot p^T)^2$$

$x_i \in \mathbb{R}$

| 用户 |   |   |   |   |   |   |   |   |   |   |   | 因素  |     |    |
|----|---|---|---|---|---|---|---|---|---|---|---|-----|-----|----|
| 1  |   | 3 |   |   | 5 |   |   | 5 |   | 4 |   | .1  | -.4 | .2 |
|    |   | 5 | 4 |   |   | 4 |   |   | 2 | 1 | 3 | -.5 | .6  | .5 |
| 2  | 4 |   | 1 | 2 |   | 3 |   | 4 | 3 | 5 |   | -.2 | .3  | .5 |
|    | 2 | 4 |   | 5 |   |   | 4 |   |   | 2 |   | 1.1 | 2.1 | .3 |
|    |   | 4 | 3 | 4 | 2 |   |   |   |   | 2 | 5 | -.7 | 2.1 | -2 |
| 1  |   | 3 |   | 3 |   |   | 2 |   |   | 4 |   | -1  | .7  | .3 |

~

| 用户  |     |    |     |     |     |     |     |     |     |     |     | 因素 |  |  |
|-----|-----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|--|--|
| 1.1 | -.2 | .3 | .5  | -.2 | -.5 | .8  | -.4 | .3  | 1.4 | 2.4 | -.9 |    |  |  |
| -.8 | .7  | .5 | 1.4 | .3  | -1  | 1.4 | 2.9 | -.7 | 1.2 | -.1 | 1.3 |    |  |  |
| 2.1 | -.4 | .6 | 1.7 | 2.4 | .9  | -.3 | .4  | .8  | .7  | -.6 | .1  |    |  |  |

$P^T$

$Q$

# 回到我们的问题

□ 希望将未见测试数据的SSE降至最低

□ 想法：最小化训练数据的SSE

■ 希望大 $k$ （因子数）能够捕捉到所有信号

■ 但是，对于 $k > 2$ ，SSE在测试数据中开始上升

□ 这是一个过拟合的经典例子：



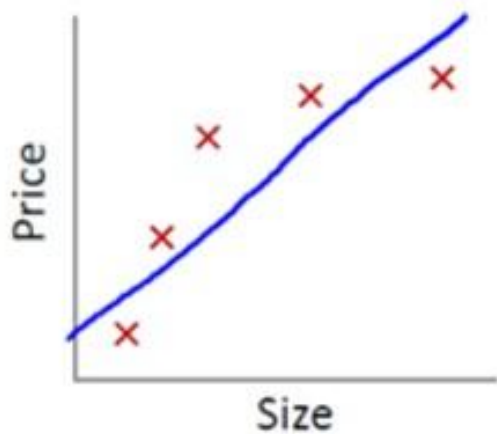
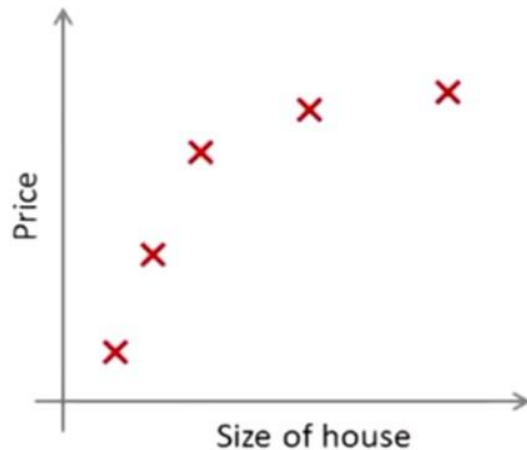
|   |   |   |   |   |  |   |  |   |  |
|---|---|---|---|---|--|---|--|---|--|
| 1 | 3 | 4 |   |   |  |   |  |   |  |
|   | 3 | 5 |   |   |  |   |  | 5 |  |
|   |   | 4 | 5 |   |  |   |  | 5 |  |
|   |   |   | 3 |   |  |   |  |   |  |
|   |   |   | 3 |   |  |   |  |   |  |
| 2 |   |   |   | ? |  |   |  |   |  |
|   |   |   |   |   |  | ? |  |   |  |
|   | 2 | 1 |   |   |  |   |  | ? |  |
|   | 3 |   |   |   |  | ? |  |   |  |
| 1 |   |   |   |   |  |   |  |   |  |

■ 如果自由度太大（自由参数太多），模型开始拟合噪声

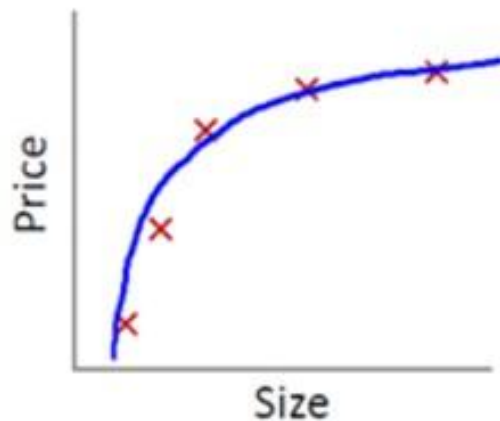
□ 这是因为该模型与训练数据匹配度太高，因此无法很好地推广到未见过的测试数据

# 过度拟合

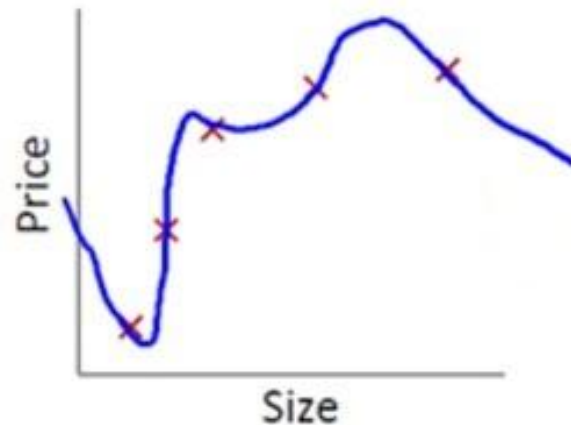
## □ 示例：房价



$$\theta_0 + \theta_1 x$$



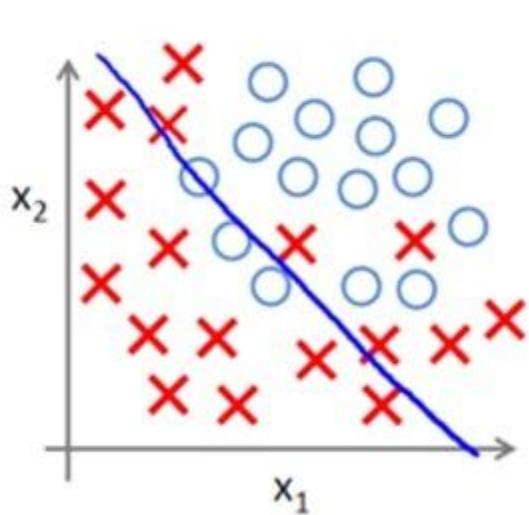
$$\theta_0 + \theta_1 x + \theta_2 x^2$$



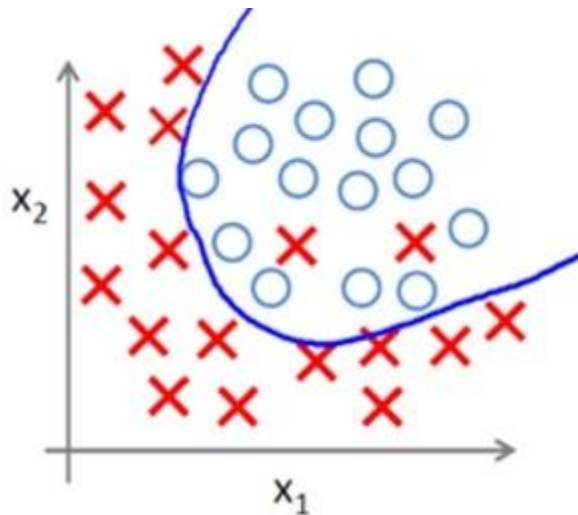
$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

# 过度拟合

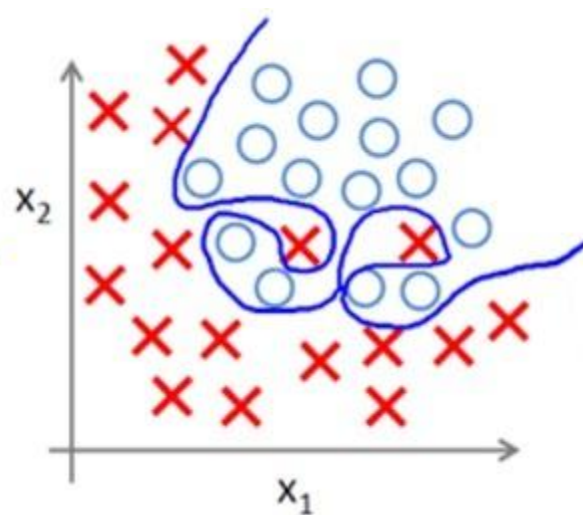
---



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

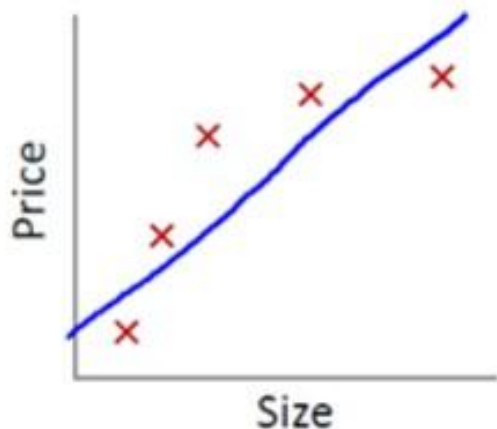


$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$



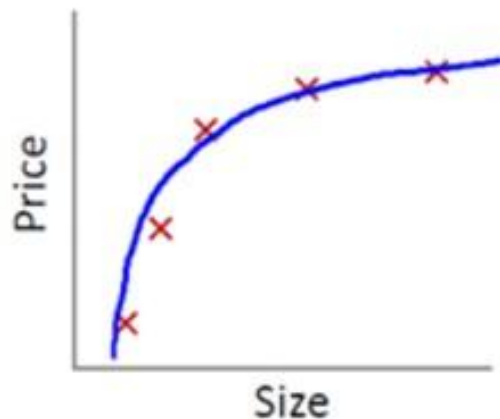
# 过度拟合

## □ 示例：房价

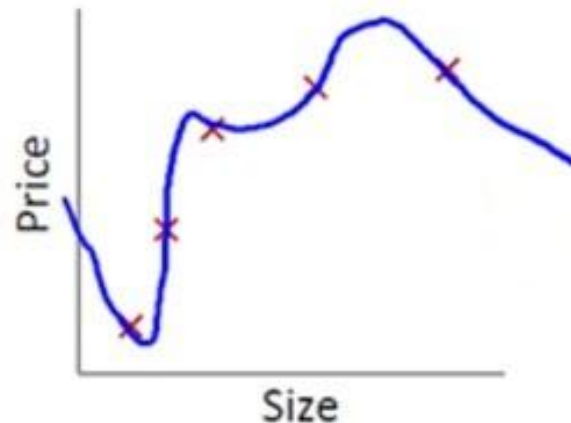


$$\theta_0 + \theta_1 x$$

下部接合处



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

过量装配

$$\min \sum_{j=1}^n (\hat{y}_{\theta} - y)^2 + \sum \lambda_j \theta^2$$

正则化（惩罚）

# 处理缺失条目

□ 为了解决过拟合问题，我们引入了  
**正则化**

- 允许使用数据充足的丰富模型
- 在数据稀缺的地方进行积极缩减

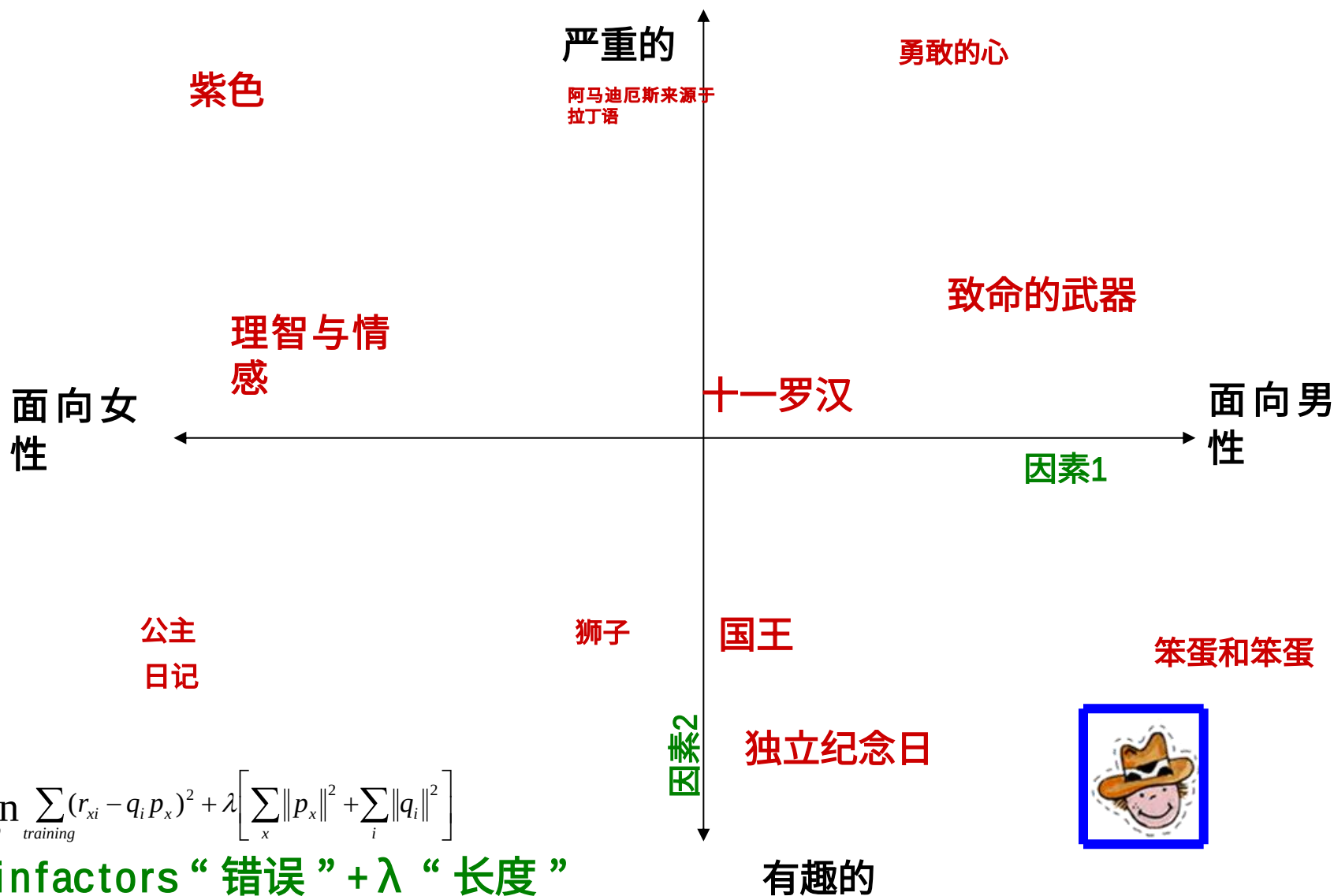
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 3 | 4 |   |   |   |
|   | 3 | 5 |   |   | 5 |
|   |   | 4 | 5 |   | 5 |
|   |   | 3 |   |   |   |
|   |   | 3 |   |   |   |
| 2 |   |   | ? |   | ? |
|   |   |   |   | ? |   |
|   | 2 | 1 |   |   | ? |
|   | 3 |   |   | ? |   |
| 1 |   |   |   |   |   |

$$\min_{P,Q} \underbrace{\sum_{training} (r_{xi} - q_i p_x)^2}_{\text{"error"}} + \underbrace{\left[ \lambda_1 \sum_x \|p_x\|^2 + \lambda_2 \sum_i \|q_i\|^2 \right]}_{\text{"length"}}$$

$\lambda_1$ 、 $\lambda_2$ ……用户设置的正则化参数

注：我们不关心目标函数的“原始”值，但我们关心的是P、Q是否达到目标函数的最小值

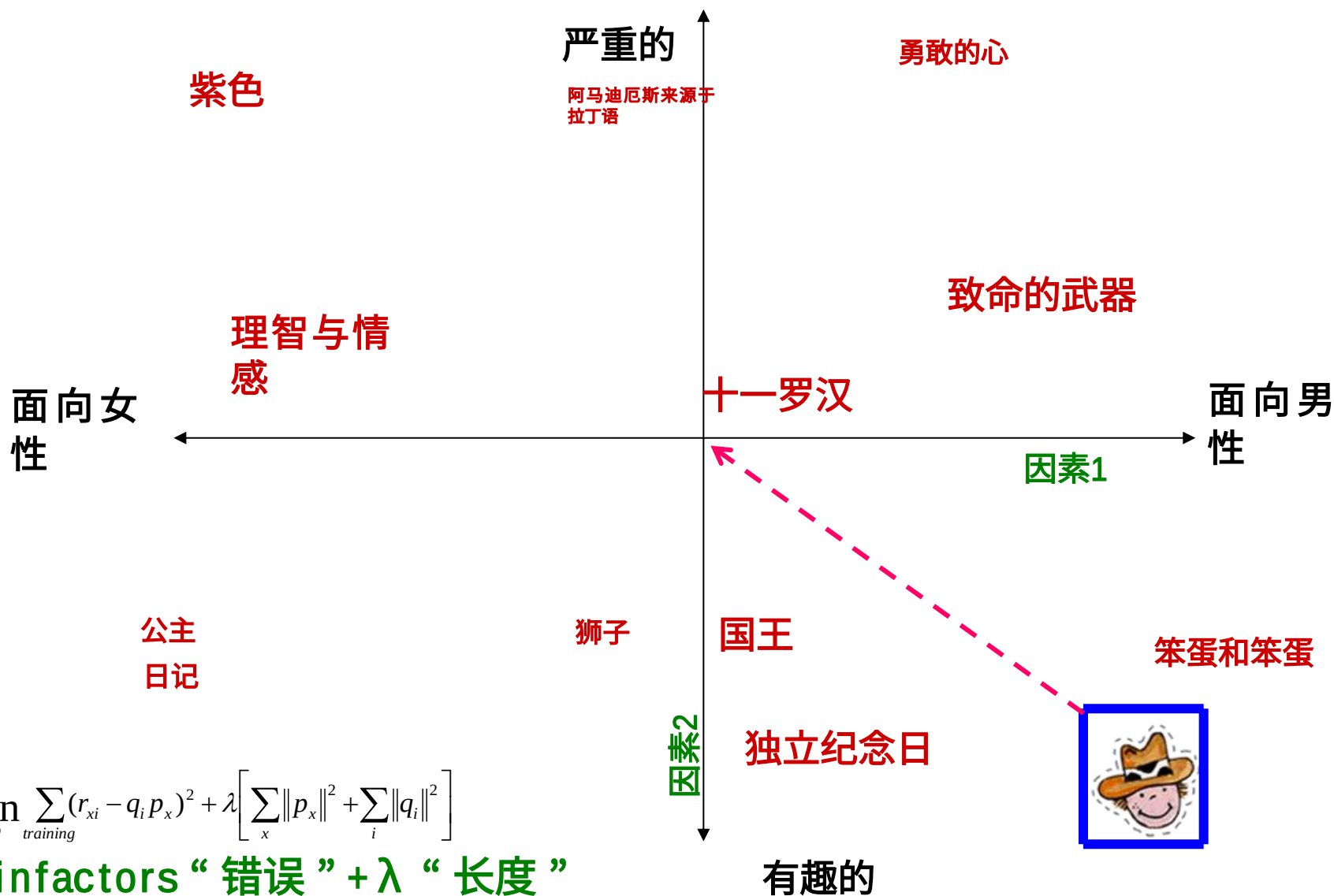
# 正则化的影响



$$\min_{P, Q} \sum_{\text{training}} (r_{xi} - q_i p_x)^2 + \lambda \left[ \sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

minfactors “错误” +  $\lambda$  “长度”

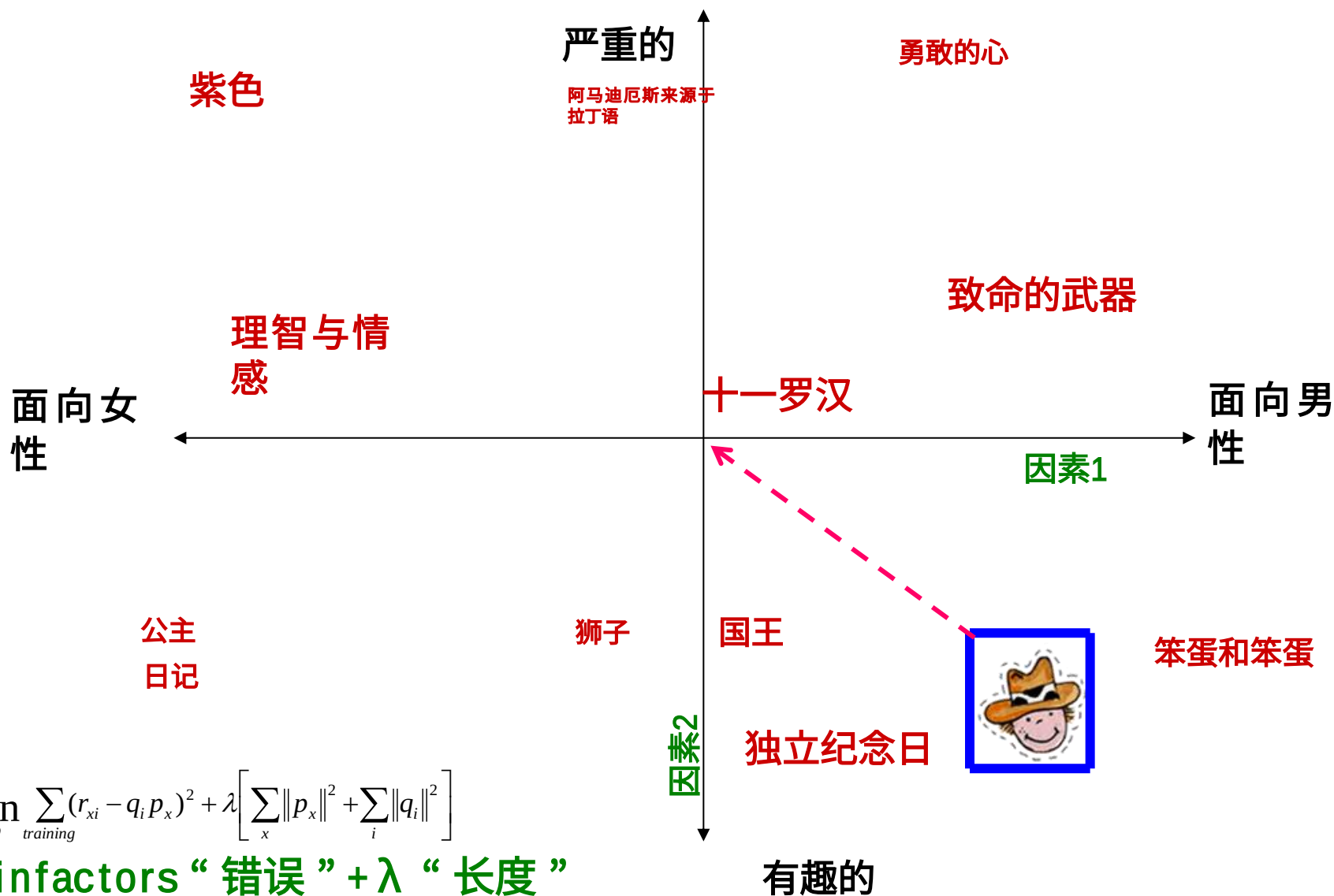
# 正则化的影响



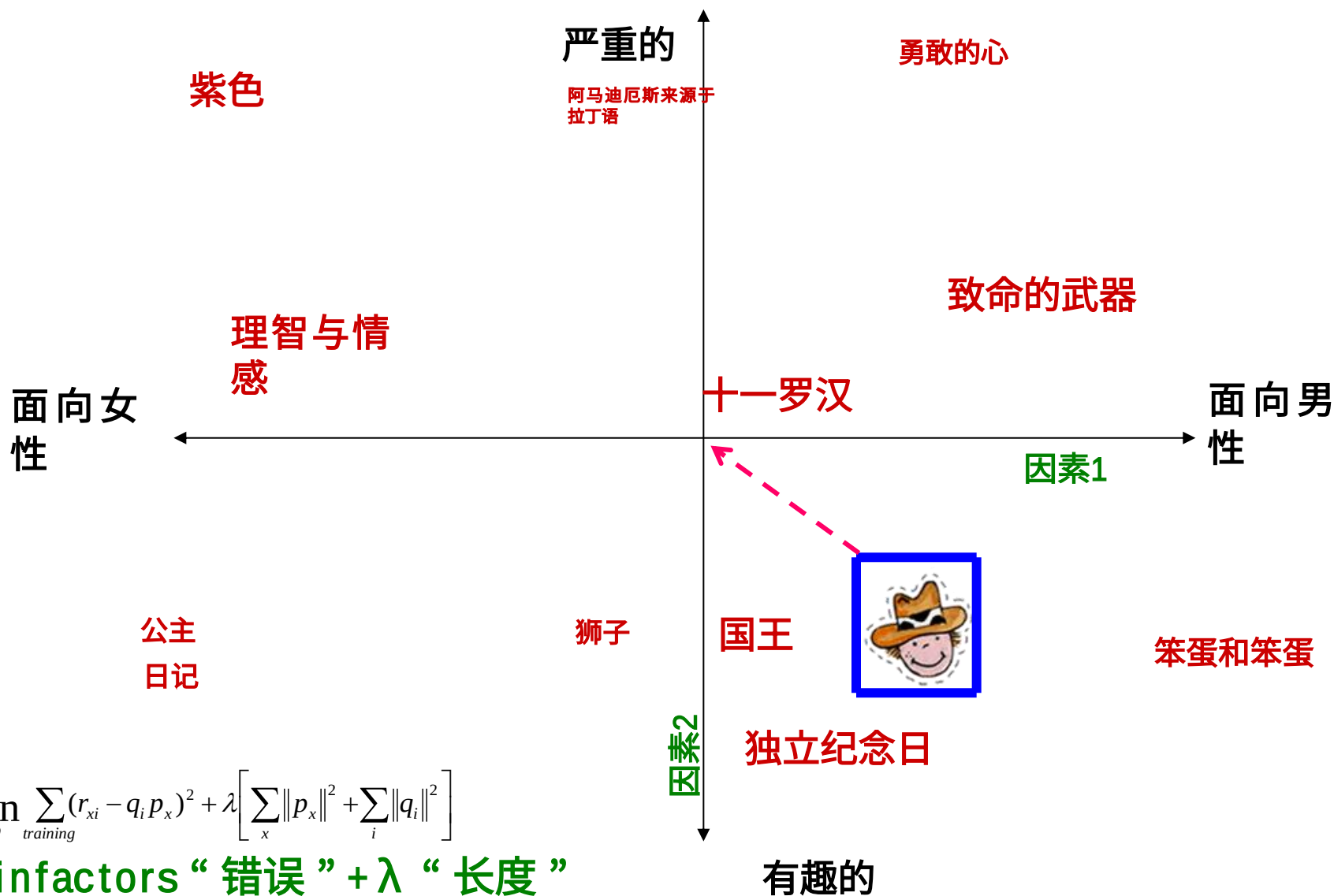
$$\min_{P, Q} \sum_{\text{training}} (r_{xi} - q_i p_x)^2 + \lambda \left[ \sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

minfactors “错误” +  $\lambda$  “长度”

# 正则化的影响



# 正则化的影响



$$\min_{P, Q} \sum_{training} (r_{xi} - q_i p_x)^2 + \lambda \left[ \sum_x \|p_x\|^2 + \sum_i \|q_i\|^2 \right]$$

minfactors “错误” +  $\lambda$  “长度”

# 梯度下降

## □ 想要找到矩阵P和Q：

$$\min_{P, Q} \sum_{\text{training}} (r_{xi} - q_i p_x)^2 + \left[ \lambda_1 \sum_x \|p_x\|^2 + \lambda_2 \sum_i \|q_i\|^2 \right]$$

## □ 梯度下降：

- 初始化P和Q（使用SVD，假装缺失的评分是0）

- 采用梯度下降法：

- $P \leftarrow P - \eta \cdot \nabla P$

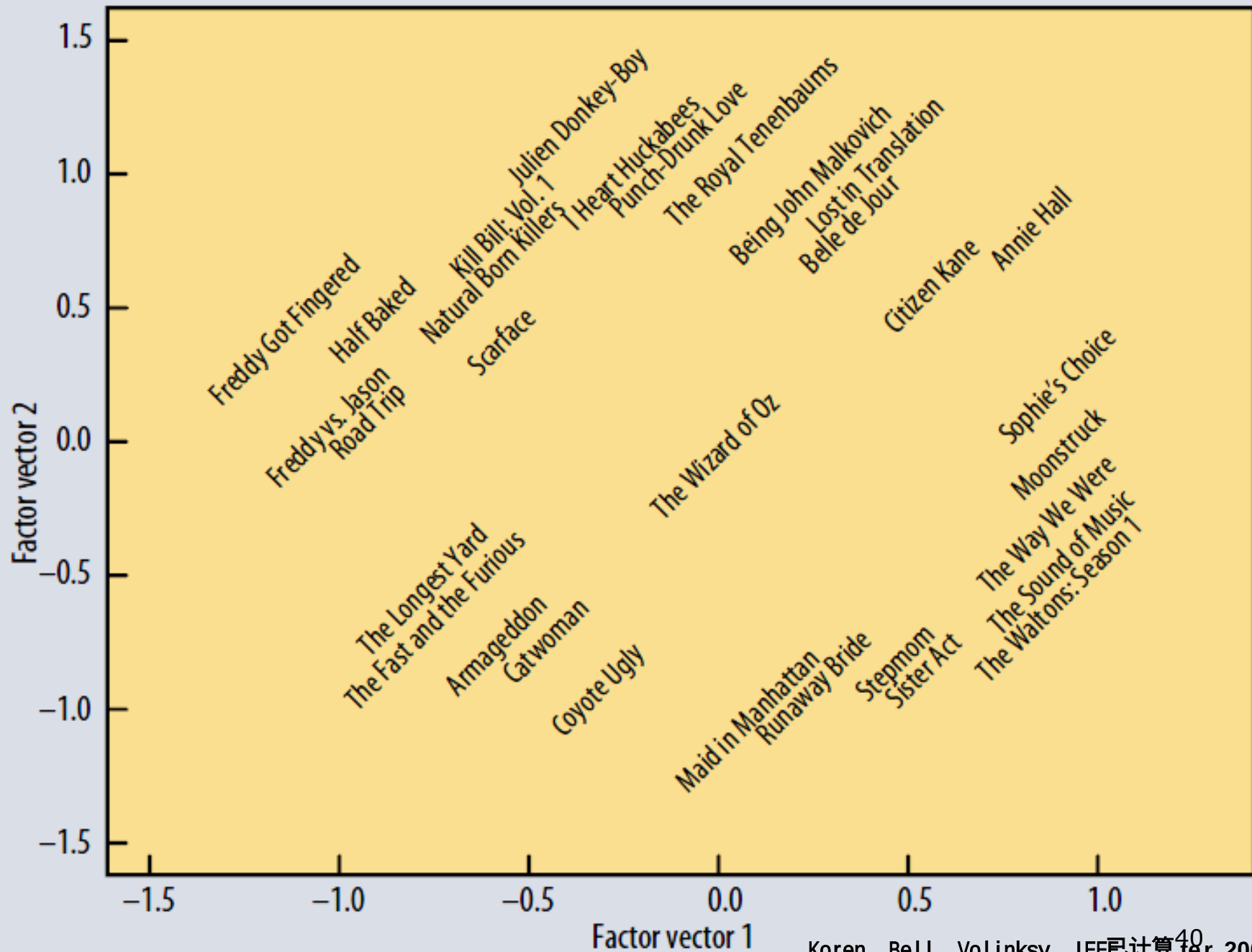
- $Q \leftarrow Q - \eta \cdot \nabla Q$

- 其中  $\nabla Q$  是矩阵Q的梯度/导数：

- $\nabla Q = [\nabla q_i f]$  和  $\nabla q_i f = \sum_x (r_{xi} - q_i p_x) p_x f + 2\lambda_2 q_i f$

- 这里  $q_i f$  是矩阵Q的第  $q_i$  行的元素

How a matrix compute 梯度  
Compute gradient of every  
element independently!



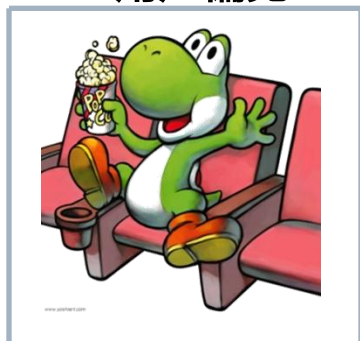


# 扩展潜在因子模型到 包含偏差

---

# 建模偏倚和相互作用

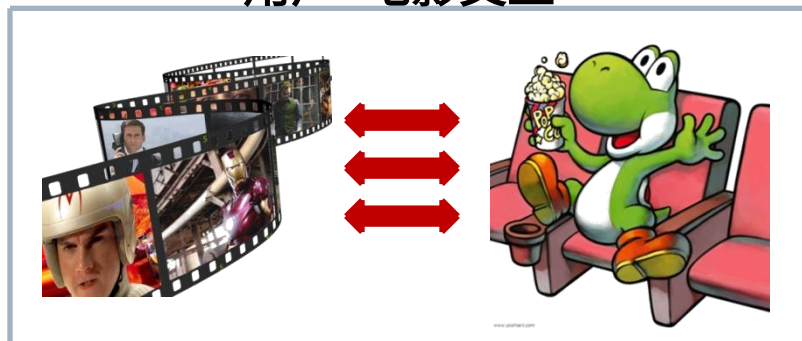
用户偏见



电影偏见



用户-电影交互



## 基线预测值

- 分离用户和电影
- 通过洞察用户行为获得收益
- 竞赛的主要实际贡献

## 用户-电影互动

描述用户与电影之间的匹配

吸引了该领域大部分的研究  
· 从算法和数学创新中获益

$\mu$  = 总体平均评分.  $b_x$  = 用户  $x$  的偏见.  $b_i$  = 电影  $i$  的偏见

# 基线预测器

---

- 我们对用户 $x$ 对电影 $i$ 的评分有预期，即使不估计 $x$ 对像 $i$ 这样的电影的态度



- 用户 $x$ 的评分量表
- 用户最近给出的其他评分值（特定日期的情绪、锚定、多用户账户）

- （最近）电影 $i$ 的流行程度
- 选择偏倚；与用户在同一天给出的评分数量相关（“频率”）

# 综合考虑

---

$$r_{xi} = \underbrace{\mu}_{\text{全面的}} + \underbrace{b_x}_{\text{偏倚来源}} + \underbrace{b_i}_{\text{偏倚来源}} + \underbrace{q_i \cdot p_x}_{\text{用户-电影}}$$

平均评分用户x电影i互相影响

## □ 样例

- 平均评分：  $\mu = 3.7$
- 您是一位关键评论者：您的评分比平均值低1星： $b_x = -1$
- 《星球大战》的评分比一般电影高出0.5分： $b_i = + 0.5$
- 《星球大战》中你获得的评分预测：  
 **$= 3.7 - 1 + 0.5 = 3.2$**

# 安装新模型

---

## □ 解决：

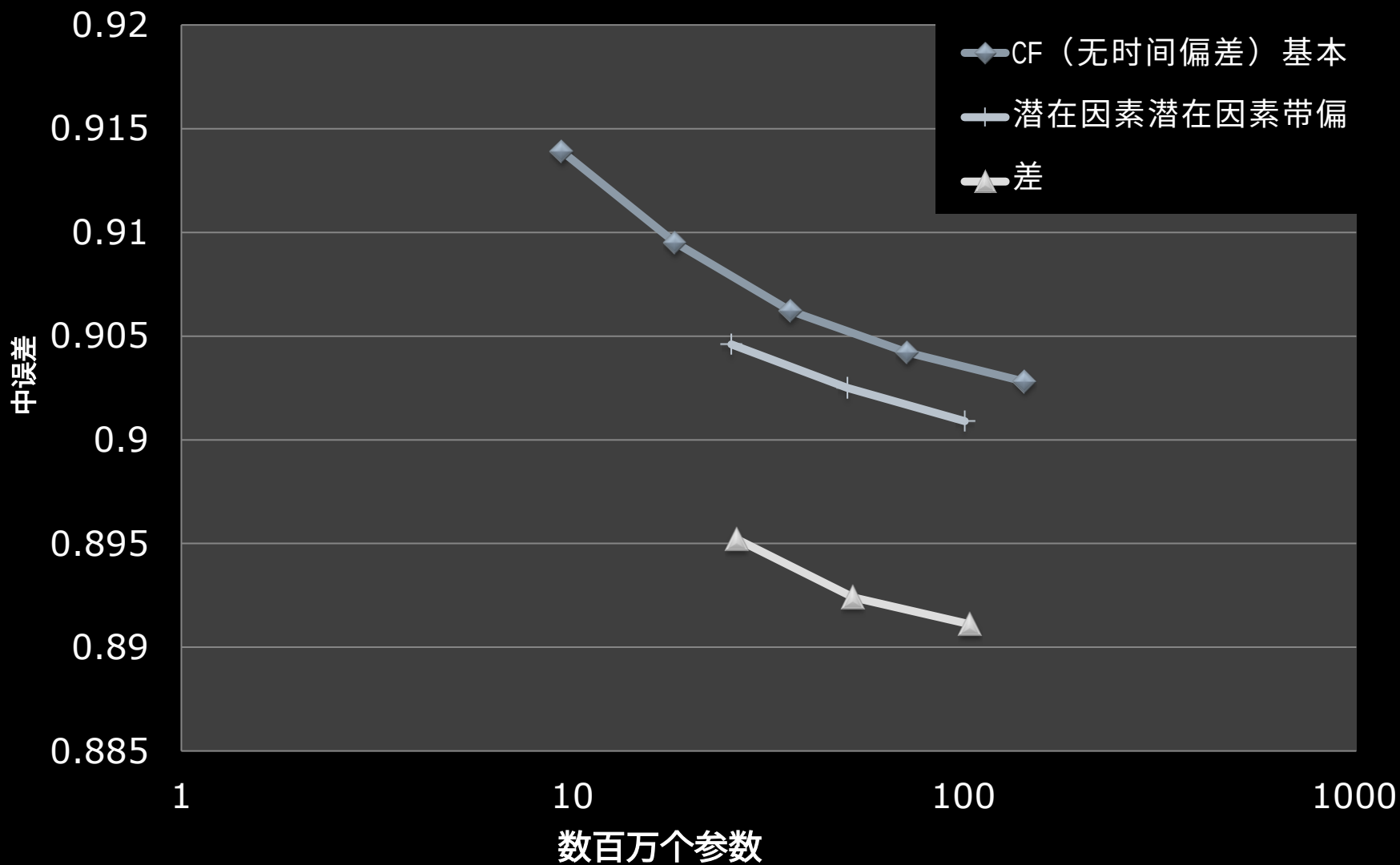
$$\min_{Q,P} \sum_{(x,i) \in R} \underbrace{\left( r_{xi} - (\mu + b_x + b_i + q_i p_x) \right)^2}_{\text{goodness of fit}} + \underbrace{\left( \lambda_1 \sum_i \|q_i\|^2 + \lambda_2 \sum_x \|p_x\|^2 + \lambda_3 \sum_x \|b_x\|^2 + \lambda_4 \sum_i \|b_i\|^2 \right)}_{\text{regularization}}$$

通过在验证集上进行网格  
搜索来选择 $\lambda$

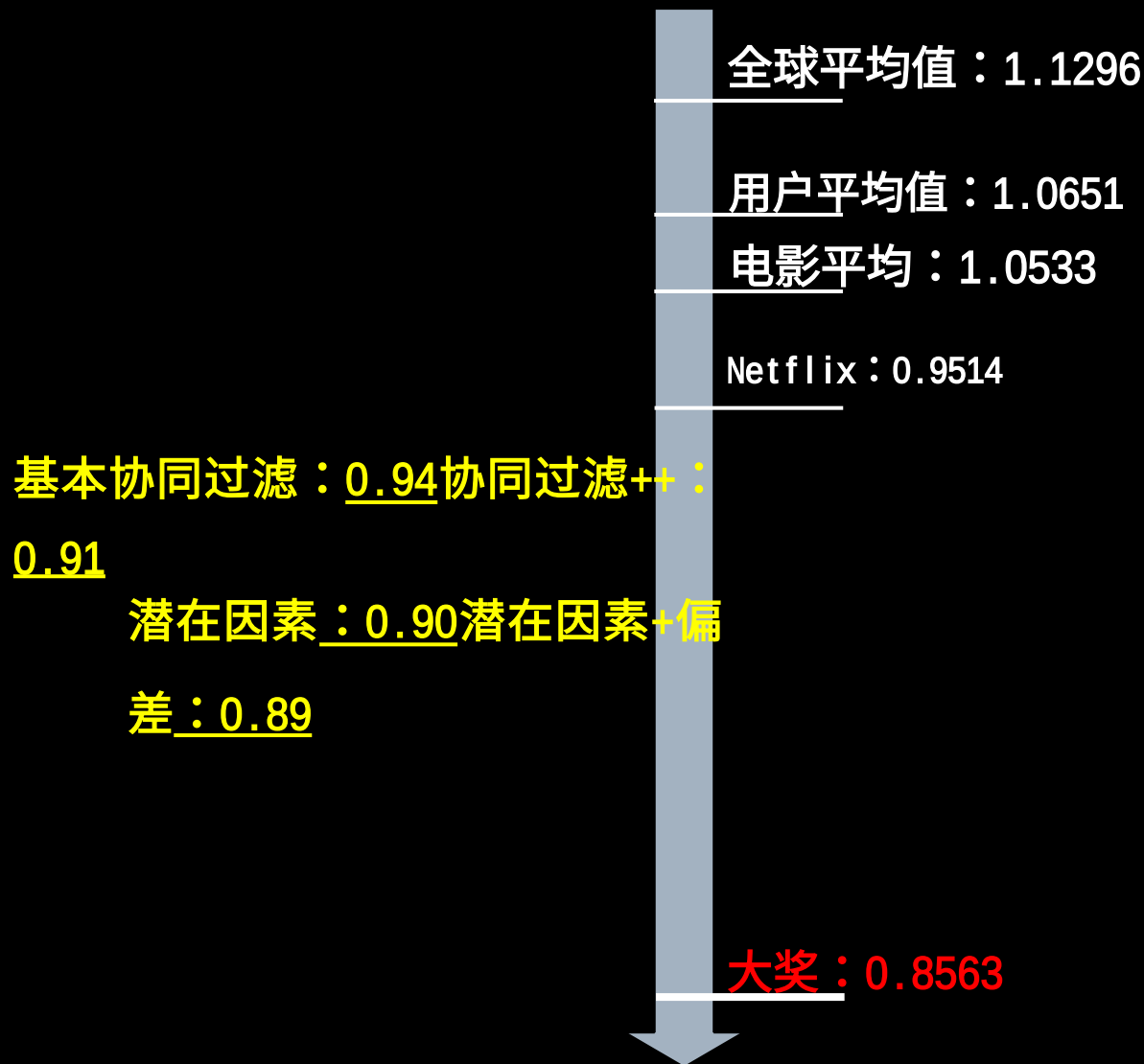
## □ (随机) 梯度下降法求参数

- 注：偏倚 $b_x$ 、 $b_i$ 以及相互作用 $q_i$ 、 $p_x$ 均被视为参数（我们对其进行估计）

# 各种方法的性能



# 各种方法的性能



# Netflix挑战：2006-2009年

---



# 用户的时间偏差

## □ 电影平均评分的突然上升

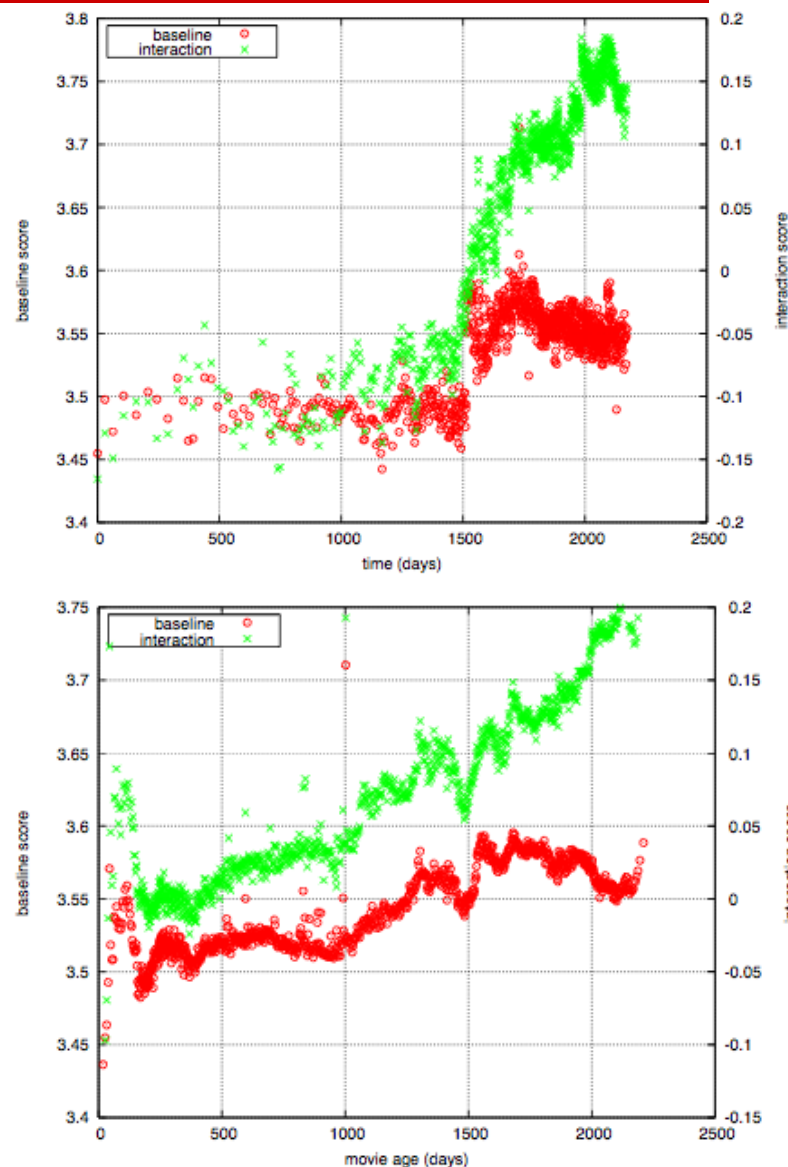
(2004年初)

- Netflix的改进
- GUI改进
- 评级含义变更

## □ 电影年龄

- 用户喜欢没有理由的新电影
- 老电影天生就比新电影好

Y. Koren, 具有时间动态的协同过滤,  
KDD, 09



# 时间偏倚和因素

---

## □ 原型号：

$$r_{xi} = m + b_x + b_i + q_i \cdot p_x$$

## □ 增加偏差的时间依赖性：

$$r_{xi} = m + b_x(t) + b_i(t) + q_i \cdot p_x$$

- 使参数 $b_x$ 和 $b_i$ 取值于时间
- (1) 通过线性趋势参数化时间依赖性
- (2) 每个箱对应10个连续周

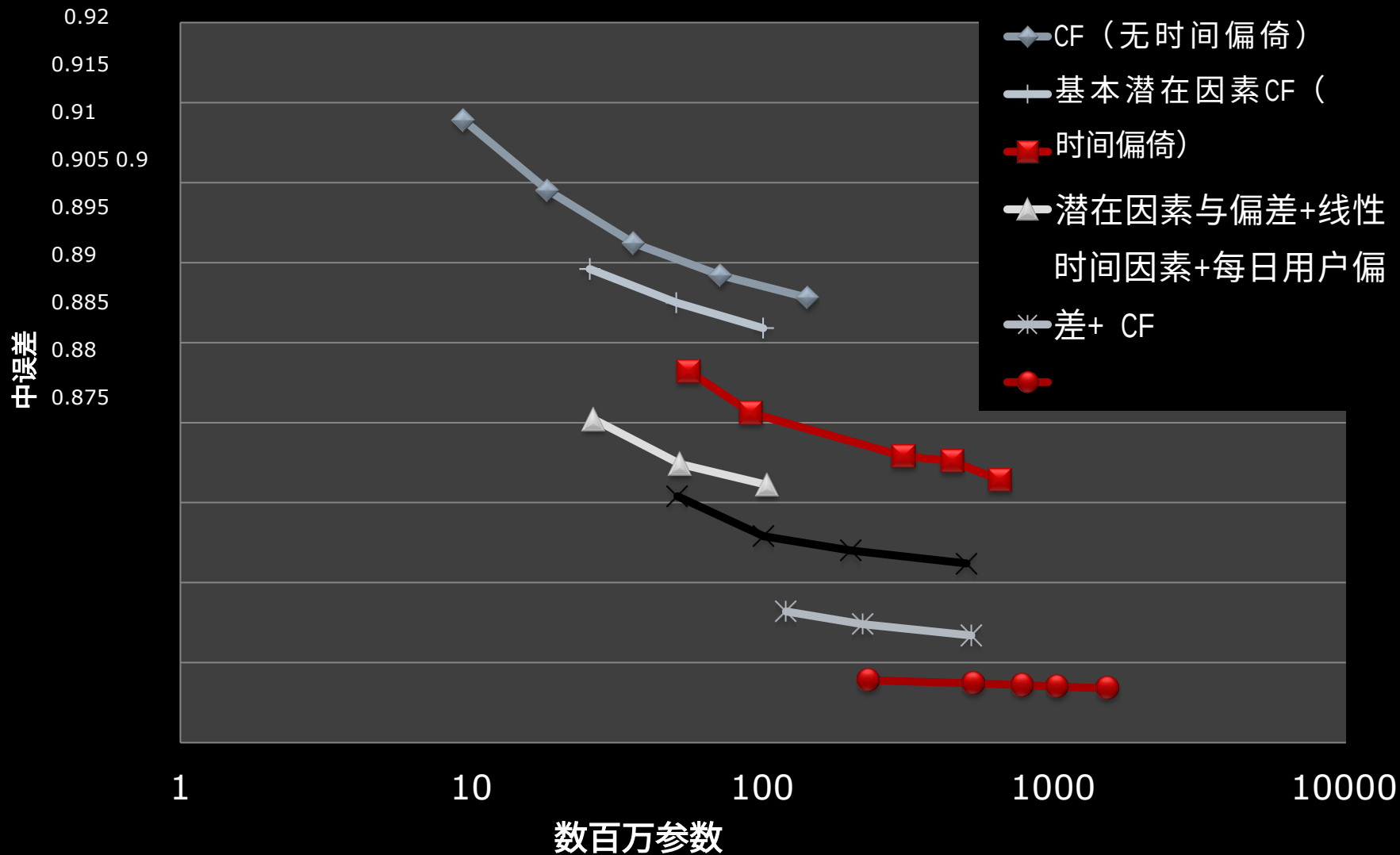
$$b_i(t) = b_i + b_{i, \text{Bin}(t)}$$

## □ 将时间依赖性添加到因素中

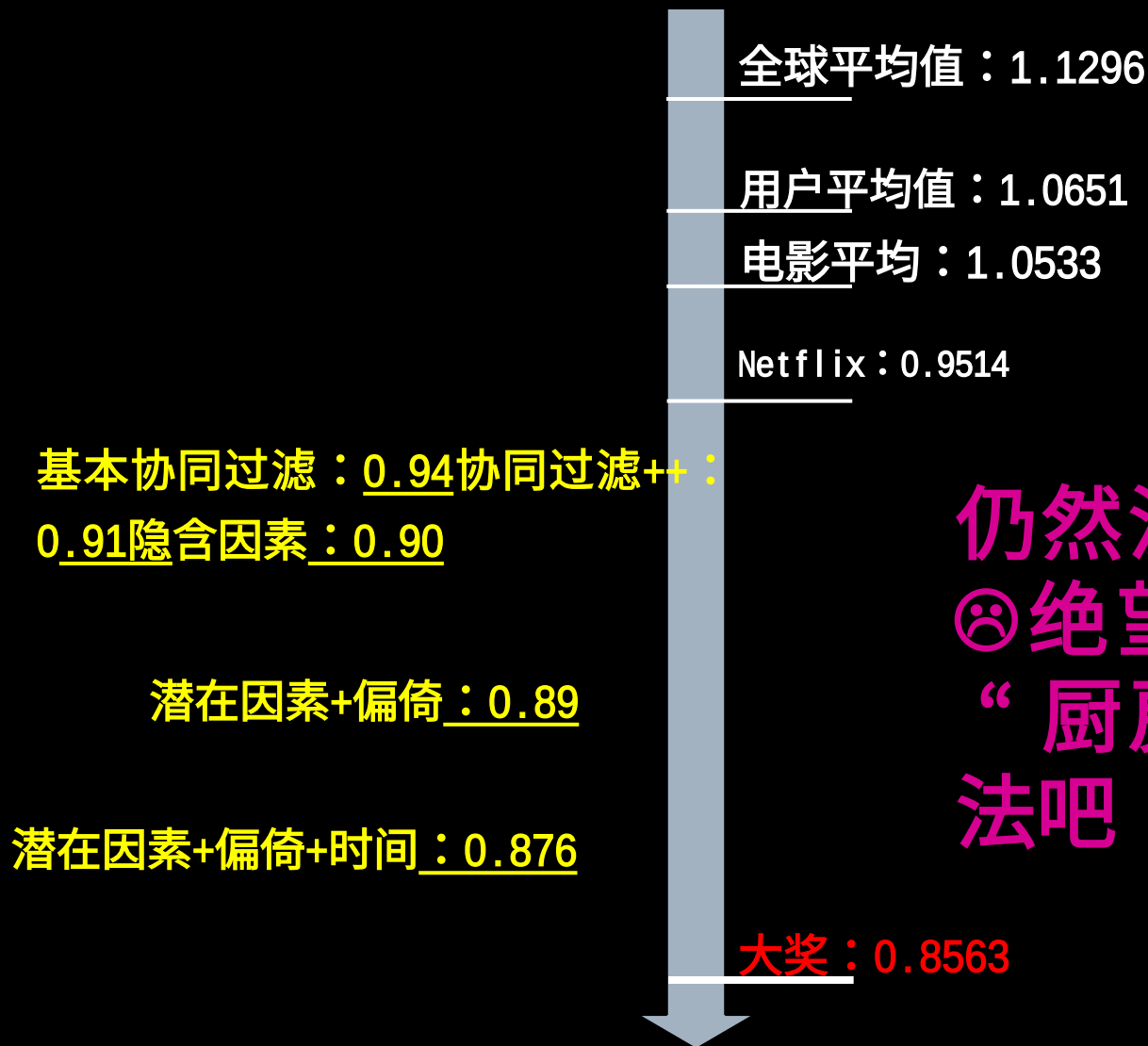
- $p_x(t)$ ... 第 $t$ 天的用户偏好向量

Y. Koren, 具有时间动态的协同过滤, KDD, 09

# 添加时间效应



# 各种方法的性能



仍然没有奖品！  
☹️ 绝望了，试试  
“厨房水槽”方法吧！

---

---

# 2009年6月26日成立

**NETFLIX**

## Netflix Prize

Home Rules Leaderboard Register Update Submit Download

### Leaderboard

Display top 20 leaders.

| Rank                                                                           | Team Name                                 | Best Score | % Improvement | Last Submit Time    |
|--------------------------------------------------------------------------------|-------------------------------------------|------------|---------------|---------------------|
| 1                                                                              | <a href="#">BellKor's Pragmatic Chaos</a> | 0.8558     | 10.05         | 2009-06-26 18:42:37 |
| <b>Grand Prize - RMSE <math>\leq</math> 0.8563</b>                             |                                           |            |               |                     |
| 2                                                                              | <a href="#">PragmaticTheory</a>           | 0.8582     | 9.80          | 2009-06-25 22:15:51 |
| 3                                                                              | <a href="#">BellKor in BigChaos</a>       | 0.8590     | 9.71          | 2009-05-13 08:14:09 |
| 4                                                                              | <a href="#">Grand Prize Team</a>          | 0.8593     | 9.68          | 2009-06-12 08:20:24 |
| 5                                                                              | <a href="#">Dace</a>                      | 0.8604     | 9.56          | 2009-04-22 05:57:03 |
| 6                                                                              | <a href="#">BigChaos</a>                  | 0.8613     | 9.47          | 2009-06-23 23:06:52 |
| <b>Progress Prize 2008 - RMSE = 0.8616 - Winning Team: BellKor in BigChaos</b> |                                           |            |               |                     |
| 7                                                                              | <a href="#">BellKor</a>                   | 0.8620     | 9.40          | 2009-06-24 07:16:02 |
| 8                                                                              | <a href="#">Gravity</a>                   | 0.8634     | 9.25          | 2009-04-22 18:31:32 |
| 9                                                                              | <a href="#">Opera Solutions</a>           | 0.8638     | 9.21          | 2009-06-26 23:18:13 |
| 10                                                                             | <a href="#">BruceDengDaoCiYiYou</a>       | 0.8638     | 9.21          | 2009-06-27 00:55:55 |
| 11                                                                             | <a href="#">pengpengzhou</a>              | 0.8638     | 9.21          | 2009-06-27 01:06:43 |
| 12                                                                             | <a href="#">xlvector</a>                  | 0.8639     | 9.20          | 2009-06-26 13:49:04 |
| 13                                                                             | xiangliang                                | 0.8639     | 9.20          | 2009-06-26 07:47:34 |
| 14                                                                             | <a href="#">Feeds2</a>                    | 0.8641     | 9.18          | 2009-06-26 22:51:55 |
| 15                                                                             | <a href="#">Ces</a>                       | 0.8642     | 9.17          | 2009-06-24 14:34:14 |

6月26日提交，触发30天“最后呼吁”

# 最后30天

---

## □ 组建团队成功

- 排行榜上的其他团队组成了一个新团队
- 依赖于组合模型
- 快速获得超过10%的合格分数

## □ 贝尔科

- 继续获得分数的小幅提升
- 意识到他们与Ensemble是直接竞争者

## □ 战略

- 两队都在密切关注排行榜
- 检查改进的唯一方法是提交一组预测
  - 这将提醒其他团队你的最新得分

# 24 距离截止时间还有几小时

---

## □ 每天提交次数限制为1次

- 在最后24小时内只能提交1份最终版本

## □ 截止日期前24小时...

- 澳大利亚BellKor团队成员偶然发现Ensemble发布的分数略高于BellKor

## □ 两队在过去的24小时里都处于疯狂状态

- 最终优化需要大量计算机时间
- 经过精心校准，可在截止时间前约一小时结束

## □ 最后的陈述

- BellKor提前提交了一份报告（有意提前），比截止时间早了40分钟
- 集合在20分钟后提交他们的最终作品
- 每个人都等着。



# Netflix Prize

COMPLETED

[Home](#) [Rules](#) [Leaderboard](#) [Update](#) [Download](#)

## Leaderboard

Showing Test Score. [Click here to show quiz score](#)Display top  leaders.

| Rank | Team Name | Best Test Score | % Improvement | Best Submit Time |
|------|-----------|-----------------|---------------|------------------|
|------|-----------|-----------------|---------------|------------------|

Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos

|    |                                                     |        |       |                     |
|----|-----------------------------------------------------|--------|-------|---------------------|
| 1  | <a href="#">BellKor's Pragmatic Chaos</a>           | 0.8567 | 10.06 | 2009-07-26 18:18:28 |
| 2  | <a href="#">The Ensemble</a>                        | 0.8567 | 10.06 | 2009-07-26 18:38:22 |
| 3  | <a href="#">Grand Prize Team</a>                    | 0.8582 | 9.98  | 2009-07-10 21:24:48 |
| 4  | <a href="#">Opera Solutions and Vandelay United</a> | 0.8588 | 9.84  | 2009-07-10 01:12:31 |
| 5  | <a href="#">Vandelay Industries!</a>                | 0.8591 | 9.81  | 2009-07-10 00:32:20 |
| 6  | <a href="#">PragmaticTheory</a>                     | 0.8594 | 9.77  | 2009-06-24 12:06:56 |
| 7  | <a href="#">BellKor in BigChaos</a>                 | 0.8601 | 9.70  | 2009-05-13 08:14:09 |
| 8  | <a href="#">Dace</a>                                | 0.8612 | 9.59  | 2009-07-24 17:18:43 |
| 9  | <a href="#">Feeds2</a>                              | 0.8622 | 9.48  | 2009-07-12 13:11:51 |
| 10 | <a href="#">BigChaos</a>                            | 0.8623 | 9.47  | 2009-04-07 12:33:59 |
| 11 | <a href="#">Opera Solutions</a>                     | 0.8623 | 9.47  | 2009-07-24 00:34:07 |
| 12 | <a href="#">BellKor</a>                             | 0.8624 | 9.46  | 2009-07-26 17:19:11 |

Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos

|    |                                        |        |      |                     |
|----|----------------------------------------|--------|------|---------------------|
| 13 | <a href="#">xiangliang</a>             | 0.8642 | 9.27 | 2009-07-15 14:53:22 |
| 14 | <a href="#">Gravity</a>                | 0.8643 | 9.26 | 2009-04-22 18:31:32 |
| 15 | <a href="#">Ces</a>                    | 0.8651 | 9.18 | 2009-06-21 19:24:53 |
| 16 | <a href="#">Invisible Ideas</a>        | 0.8653 | 9.15 | 2009-07-15 15:53:04 |
| 17 | <a href="#">Just a guy in a garage</a> | 0.8662 | 9.06 | 2009-05-24 10:02:54 |
| 18 | <a href="#">J Dennis Su</a>            | 0.8666 | 9.02 | 2009-03-07 17:16:17 |
| 19 | <a href="#">Craig Carmichael</a>       | 0.8666 | 9.02 | 2009-07-25 16:00:54 |
| 20 | <a href="#">acmehill</a>               | 0.8668 | 9.00 | 2009-03-21 16:20:50 |

Progress Prize 2007 - RMSE = 0.8723 - Winning Team: KorBell

# Million \$于2009年9月21日获得

---



# 致谢

---

- 一些幻灯片和图表借自 Yehuda Koren, Robert Bell 和 Padhraic Smyth
- 进一步阅读：
  - Y. Koren, 具有时间动态的协同过滤, KDD, 09
  - <http://www2.research.att.com/~volinsky/netflix/bpc.html>
  - <http://www.the-ensemble.com/>

# 致谢

---

## □ 幻灯片改编自：

- Jeffrey D. Ullman教授
- Anand Rajaraman博士
- Jure Leskovec博士